

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL DAN DEBUGGING

Oleh:

Muhammad Dzul Fathi Ahyan NIM. 2410817210011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dzul Fathi Ahyan

NIM : 2410817310009

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Raymond Hariyono
NIM. 2310817210007

Erika Maulidiya, S.Kom., M.Kom.
NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	7
Source Code XML	7
Source Code Compose	22
B. Output Program.....	35
C. Pembahasan.....	44
Pembahasan XML	44
Pembahasan Compose.....	68
SOAL 2.....	92
REFERENSI.....	95
TAUTAN GIT	96

DAFTAR GAMBAR

Gambar 1. Contoh Debug dari Soal.....	6
Gambar 2. Tampilan Awal Aplikasi XML (Vertikal)	35
Gambar 3. Tampilan Halaman Detail Aplikasi XML (Vertikal).....	36
Gambar 4 Tampilan Aplikasi XML (Horizontal).....	36
Gambar 5 Tampilan Halaman Detail Aplikasi XML (Horizontal).....	36
Gambar 6 Hasil Logcat dari XML stasaat Masuk Aplikasi	37
Gambar 7 Hasil Logcat dari XML saat Klik Instagram	37
Gambar 8 Hasil Logcat dari XML saat Klik Detail.....	37
Gambar 9 Break Point dan Hasil Debug XML.....	37
Gambar 10 Hasil Step Over XML	38
Gambar 11 Hasil Step Into XML	38
Gambar 12 Hasil Step Out XML	39
Gambar 13. Tampilan Awal Aplikasi Compose (Vertikal)	39
Gambar 14. Tampilan Halaman Detail Aplikasi Compose (Vertikal)	40
Gambar 15. Tampilan Aplikasi Compose (Horizontal)	40
Gambar 16. Tampilan Halaman Detail Aplikasi Compose (Horizontal)	40
Gambar 17 Hasil Logcat dari Compose saat Masuk Aplika	41
Gambar 18 Hasil Logcat dari Compose saat Klik Instagram	41
Gambar 19 Hasil Logcat dari Compose saat Klik Detail	41
Gambar 20 Break Point Compose	41
Gambar 21 Hasil Debug Compose	42
Gambar 22 Hasil Step Over Compose.....	42
Gambar 23 Hasil Step Into Compose	43
Gambar 24 Hasil Step Out Compose.....	43

DAFTAR TABEL

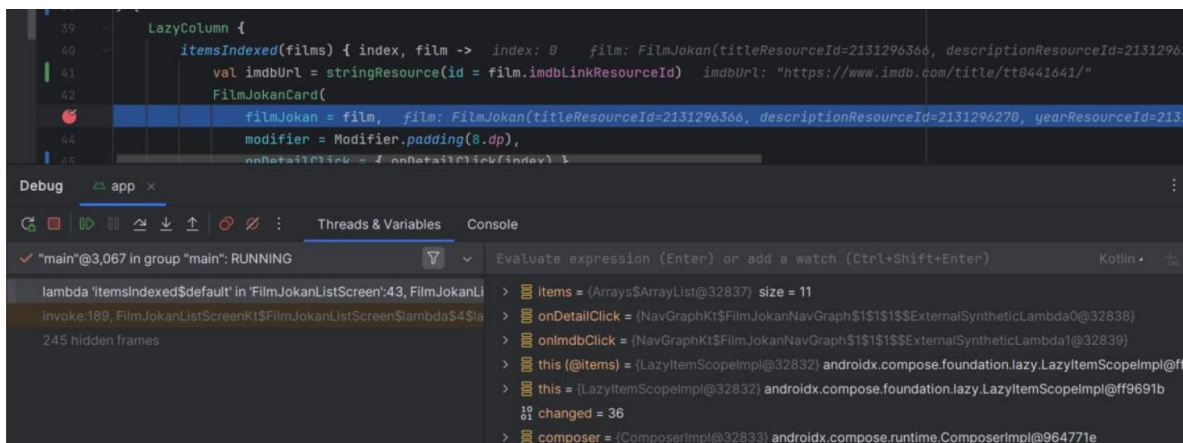
Tabel 1. Source Code FirendData Jawaban Soal 1 XML	7
Tabel 2 Source Code Friend Jawaban Soal 1 XML	8
Tabel 3 Source Code Adapter.kt Jawaban Soal 1 XML	8
Tabel 4 Source Code DetailFragment Jawaban Soal 1 XML	10
Tabel 5 Source Code FriendListFragment Jawaban Soal 1 XML	11
Tabel 6 Source Code FriendViewModel Jawaban Soal 1 XML	13
Tabel 7 Source Code MainActivity Jawaban Soal 1 XML	15
Tabel 8 Source Code MainApplication Jawaban Soal 1 XML	15
Tabel 9 Source Code fragment_detail.xml Jawaban Soal 1 XML	15
Tabel 10 Source Code fragment_friend_list.xml Jawaban Soal 1 XML	17
Tabel 11 Source Code item_friend.xml Jawaban Soal 1 XML	18
Tabel 12 Source Code nav_graph.xml Jawaban Soal 1 XML	20
Tabel 13. Source Code MainActivity.kt Jawaban Soal 1 Compose	22
Tabel 14 Source Code Friend Data Jawaban Soal 1 Compose	22
Tabel 15 Source Code Friend Jawaban Soal 1 Compose	24
Tabel 16 Source Code DetailScreen.kt Jawaban Soal 1 Compose	25
Tabel 17 Source Code FriendItem.kt Jawaban Soal 1 Compose	27
Tabel 18 Souce Code FriendListScreen.kt Jawaban Soal 1 Compose	30
Tabel 19 Source Code AppNavigation.kt Jawaban Soal 1 Compose	31
Tabel 20 Source Code FriendViewModel.kt Jawaban Soal 1 Compose	33
Tabel 21 Source Code MainApplication.kt Jawaban Soal 1 Compose	34
Tabel 22 Contoh Debugging	93

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity
- Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- Install dan gunakan library Timber untuk logging event berikut:
 - Log saat data item masuk ke dalam list
 - Log saat tombol Detail dan tombol Explicit Intent ditekan
 - Log data dari list yang dipilih ketika berpindah ke halaman Detail
- Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Debug dari Soal

A. Source Code

Source Code XML

FriendData

Tabel 1. Source Code FirendData Jawaban Soal 1 XML

1	package com.example.modul3xml.data
2	
3	import com.example.modul3xml.R
4	import com.example.modul3xml.model.Friend
5	
6	object FriendData {
7	val friends = listOf(
8	Friend(1, "Muhammad Lv Gunawuy", "32 Agustus 6969", "Foto lv ini diambil ketika dia ingin bercita cita menguasai dunia dan bertekad bahwa dia akan menjadi heavenly demon perkap yang tidak terkalahkan dan juga akan mmenentukan penerus heavenly demon perkap selanjutnya di bawah kepemimpinan dia", R.drawable.lv, "https://instagram.com/"),
9	Friend(2, "Fatih Bari Muar", "29 Masehi 3802", "Fatih merupakan seorang puitisi yang sangat tidak terkenal dan tidak akan terkenal kerna puitisi dia hanya dapat digunakan dalam sekali pakai, foto ini diambil ketika dia sedang ingin buang hajat dan ingin mencari insiparsi untuk puisi yang akan dia buat selanjutnya", R.drawable.fatih, "https://instagram.com/"),
10	Friend(3, "Meluyyyyyyy", "32 March 12738", "Foto ini ketika Meluy sedang bermain sebua aplikasi seperti ome tv dan sedang mencari para cwk cwk yang akan digoda oleh meluy tetapi dia malah bertemu seorang pria dan dihina hina oleh pria tersebut sebagai chain smoker", R.drawable.melon, "https://instagram.com/"),
11	Friend(4, "neRvers.", "43 April 21387", "Ini adalah foto seorang neRvers. ketika sedang lock in untuk melakukan ibadah, tapi disini dia sedang lock in karena setelah selesai ibadah dia ingin mencucuk pentol amang amang di muka masjid", R.drawable.nafis, "https://instagram.com/"),
12	Friend(5, "Rakha x Police", "48 Janus 2317", "Foto ini adalah foto seorang yang ingin bercita cita menjadi seorang police yaitu rakha TI 24 FT ULM WAKAHIM HMTI FT ULM yang diman foto ini dia sedang melakukan wawancara kepada calon calon anggota HMTI FT ULM dengan dia berniat untuk menggagalkan semua orang", R.drawable.rakha, "https://instagram.com/"))

13	)
14	}

Friend

Tabel 2 Source Code Friend Jawaban Soal 1 XML

1	package com.example.modul3xml.model
2	
3	import androidx.annotation.DrawableRes
4	
5	data class Friend(
6	val id: Int,
7	val name: String,
8	val date: String,
9	val feature: String,
10	@DrawableRes val photoResId: Int,
11	val instagramUrl: String
12	)

Adapter.kt

Tabel 3 Source Code Adapter.kt Jawaban Soal 1 XML

1	package com.example.modul3xml.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.view.LayoutInflater
6	import android.view.ViewGroup
7	import androidx.recyclerview.widget.RecyclerView
8	import com.bumptech.glide.Glide
9	import
	com.example.modul3xml.databinding.ItemFriendBinding
10	import com.example.modul3xml.model.Friend
11	import timber.log.Timber
12	
13	class FriendAdapter(
14	private val friends: List<Friend>,
15	private val onDetailClick: (Friend) -> Unit // Ubah
	tipe menjadi Friend
16	)
	RecyclerView.Adapter<FriendAdapter.FriendViewHolder>() {
17	
18	inner class FriendViewHolder(private val binding:
	ItemFriendBinding)
	RecyclerView.ViewHolder(binding.root) {

```

19
20         fun bind(friend: Friend) {
21             binding.tvName.text = friend.name
22             binding.tvDate.text = friend.date
23             binding.tvFeature.text = "Feature:
24             ${friend.feature}"
25
26             Glide.with(itemView.context)
27                 .load(friend.photoResId)
28                 .into(binding.ivPhoto)
29
30             // Mengatur tombol Instagram dengan TIMBER
31             LOG
32             binding.btnInstagram.setOnClickListener {
33                 Timber.d("Tombol Explicit Intent
34                 (Instagram) ditekan. Membuka URL:
35                 ${friend.instagramUrl}")
36                 val intent = Intent(Intent.ACTION_VIEW,
37                 Uri.parse(friend.instagramUrl))
38                 itemView.context.startActivity(intent)
39             }
40
41             // Memicu navigasi dengan membawa objek
42             friend
43             binding.btnDetail.setOnClickListener {
44                 onDetailClick(friend)
45             }
46         }
47
48         override fun onCreateViewHolder(parent: ViewGroup,
49         viewType: Int): FriendViewHolder {
50             val binding =
51             ItemFriendBinding.inflate(LayoutInflater.from(parent.co
52             ntext), parent, false)
53             return FriendViewHolder(binding)
54         }
55
56         override fun getItemCount(): Int = friends.size
57
58         override fun onBindViewHolder(holder:
59         FriendViewHolder, position: Int) {
60             holder.bind(friends[position])
61         }
62     }
63 }

```

DetailFragment

Tabel 4 Source Code DetailFragment Jawaban Soal 1 XML

```
1 package com.example.modul3xml.ui
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import com.bumptech.glide.Glide
9 import com.example.modul3xml.data.FriendData
10 import
11     com.example.modul3xml.databinding.FragmentDetailBinding
12
13 class DetailFragment : Fragment() {
14     private var _binding: FragmentDetailBinding? = null
15     private val binding get() = _binding!!
16
17     override fun onCreateView(
18         inflater: LayoutInflater, container: ViewGroup?,
19         savedInstanceState: Bundle?
20     ): View {
21         _binding
22         =
23         FragmentDetailBinding.inflate(inflater, container,
24         false)
25         return binding.root
26     }
27
28     override fun onViewCreated(view: View,
29         savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         // Mengambil ID dari argumen navigasi (Bundle)
33         val friendId = arguments?.getInt("friendId") ?:
34         0
35
36         // Mencari data berdasarkan ID di FriendData
37         val friend = FriendData.friends.find { it.id ==
38         friendId }
39
40         if (friend != null) {
41             binding.tvDetailName.text = friend.name
42             binding.tvDetailDate.text = friend.date
43             binding.tvDetailFeature.text
44             =
45             friend.feature
46         }
47     }
48 }
```

38	
39	Glide.with(this)
40	.load(friend.photoResId)
41	.into(binding.ivDetailPhoto)
42	} else {
43	binding.tvDetailName.text = "Data tidak
	ditemukan"
44	}
45	}
46	
47	override fun onDestroyView() {
48	super.onDestroyView()
49	_binding = null
50	}
51	}

FriendListFragment

Tabel 5 Source Code FriendListFragment Jawaban Soal 1 XML

1	package com.example.modul3xml.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.lifecycle.Lifecycle
10	import androidx.lifecycle.LifecycleScope
11	import androidx.lifecycle.repeatOnLifecycle
12	import androidx.navigation.fragment.findNavController
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import com.example.modul3xml.R
15	import
	com.example.modul3xml.databinding.FragmentFriendListBin
	ding
16	import kotlinx.coroutines.launch
17	
18	class FriendListFragment : Fragment() {
19	
20	private var _binding: FragmentFriendListBinding? =
	null
21	private val binding get() = _binding!!
22	
23	// Inisialisasi ViewModel menggunakan Factory

```

24     private val viewModel: FriendViewModel by viewModels
25     {
26         FriendViewModelFactory("Fathi")
27     }
28     override fun onCreateView(
29         inflater: LayoutInflater, container: ViewGroup?,
30         savedInstanceState: Bundle?
31     ): View {
32         _binding =
33         FragmentFriendListBinding.inflate(inflater, container,
34         false)
35         return binding.root
36     }
37     override fun onViewCreated(view: View,
38         savedInstanceState: Bundle?) {
39         super.onViewCreated(view, savedInstanceState)
40         // Mengamati StateFlow dari ViewModel
41         viewLifecycleOwner.lifecycleScope.launch {
42             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.ST
43             ARTED) {
44                 viewModel.friendsState.collect {
45                     friendsList ->
46                     // Setup Featured Friends
47                     (Horizontal) - ambil 5 teman pertama
48                     binding.rvFeaturedFriends.layoutManager =
49                     LinearLayoutManager(requireContext(),
50                     LinearLayoutManager.HORIZONTAL, false)
51                     binding.rvFeaturedFriends.adapter =
52                     FriendAdapter(friendsList.take(5)) { selectedFriend ->
53                         viewModel.onFriendClicked(selectedFriend) // Panggil log
54                         ViewModel
55                         // Navigasi ke Detail Fragment
56                         membawa ID
57                         val bundle = Bundle().apply {
58                             putInt("friendId", selectedFriend.id) }

```

```

53 findNavController().navigate(R.id.action_friendListFrag
54     ment_to_detailFragment, bundle)
55     }
56     // Setup All Friends (Vertical)
57     binding.rvAllFriends.layoutManager =
58     LinearLayoutManager(requireContext())
59     binding.rvAllFriends.adapter      =
60     FriendAdapter(friendsList) { selectedFriend ->
61
62         viewModel.onFriendClicked(selectedFriend) // Panggil log
63         ViewModel
64
65         // Navigasi ke Detail Fragment
66         membawa ID
67         val bundle = Bundle().apply {
68             putInt("friendId", selectedFriend.id) }
69
70         findNavController().navigate(R.id.action_friendListFrag
71         ment_to_detailFragment, bundle)
72     }
73     }
74     }
75     }
76
77     override fun onDestroyView() {
78         super.onDestroyView()
79         _binding = null
80     }
81 }

```

FriendViewModel.kt

Tabel 6 Source Code FriendViewModel Jawaban Soal 1 XML

```

1 package com.example.modul3xml.ui
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import com.example.modul3xml.data.FriendData
6 import com.example.modul3xml.model.Friend
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.asStateFlow

```

```

10 import timber.log.Timber
11
12 class FriendViewModel(private val userName: String) :
    ViewModel() {
13
14     // StateFlow untuk daftar teman
15     private val _friendsState =
    MutableStateFlow<List<Friend>>(emptyList())
16     val friendsState: StateFlow<List<Friend>> =
    _friendsState.asStateFlow()
17
18     init {
19         loadFriends()
20         Timber.d("ViewModel diinisialisasi oleh:
    $userName") // Syarat Log
21     }
22
23     private fun loadFriends() {
24         val data = FriendData.friends
25         _friendsState.value = data
26         Timber.d("Data item masuk ke dalam list:
    ${data.size} kawan") // Syarat Log
27     }
28
29     // Fungsi log saat tombol ditekan
30     fun onFriendClicked(friend: Friend) {
31         Timber.d("Button Detail ditekan untuk:
    ${friend.name}") // Syarat Log
32         Timber.d("Berpindah ke halaman Detail dengan
    data: $friend") // Syarat Log
33     }
34 }
35
36 class FriendViewModelFactory(private val name: String) :
    ViewModelProvider.Factory {
37     override fun <T : ViewModel> create(modelClass:
    Class<T>): T {
38         if
    (modelClass.isAssignableFrom(FriendViewModel::class.java)) {
39             @Suppress("UNCHECKED_CAST")
40             return FriendViewModel(name) as T
41         }
42         throw IllegalArgumentException("Unknown
    ViewModel class")
43     }
44 }

```

MainActivity

Tabel 7 Source Code MainActivity Jawaban Soal 1 XML

1	package com.example.modul3xml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	
6	class MainActivity : AppCompatActivity() {
7	override fun onCreate(savedInstanceState: Bundle?) {
8	super.onCreate(savedInstanceState)
9	setContentView(R.layout.activity_main)
10	}
11	}

MainApplication

Tabel 8 Source Code MainApplication Jawaban Soal 1 XML

1	package com.example.modul3xml
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MainApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	
10	// Menyalakan Timber khusus untuk mode Debug
11	val isDebuggable = (applicationInfo.flags and
	android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE) != 0
12	if (isDebuggable) {
13	Timber.plant(Timber.DebugTree())
14	}
15	}
16	}

Fragment_detail.xml

Tabel 9 Source Code fragment_detail.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"

```

4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      tools:context=".ui.DetailFragment">
8
9      <LinearLayout
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:orientation="vertical"
13         android:padding="16dp">
14
15
16         <com.google.android.material.imageview.ShapeableImageVie
17         w
18             android:id="@+id/ivDetailPhoto"
19             android:layout_width="match_parent"
20             android:layout_height="350dp"
21             android:scaleType="centerCrop"
22             app:shapeAppearanceOverlay="@style/RoundedImage" />
23
24         <TextView
25             android:id="@+id/tvDetailName"
26             android:layout_width="wrap_content"
27             android:layout_height="wrap_content"
28             android:layout_marginTop="16dp"
29             android:textSize="28sp"
30             android:textStyle="bold"
31             tools:text="Nama Teman" />
32
33         <TextView
34             android:id="@+id/tvDetailDate"
35             android:layout_width="wrap_content"
36             android:layout_height="wrap_content"
37             android:layout_marginTop="4dp"
38             android:layout_marginBottom="16dp"
39             android:textSize="16sp"
40             android:textColor="#666666"
41             tools:text="Tanggal" />
42
43         <TextView
44             android:layout_width="wrap_content"
45             android:layout_height="wrap_content"
46             android:text="Feature:"
47             android:textSize="18sp"
48             android:textStyle="bold" />

```

48	<TextView
49	android:id="@+id/tvDetailFeature"
50	android:layout_width="match_parent"
51	android:layout_height="wrap_content"
52	android:layout_marginTop="4dp"
53	android:textSize="16sp"
54	android:lineSpacingExtra="8sp"
55	tools:text="Deskripsi lengkap teman akan
56	muncul di sini" />
57	</LinearLayout>
58	</ScrollView>

Fragment_friend_list.xml

Tabel 10 Source Code fragment_friend_list.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.core.widget.NestedScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".ui.FriendListFragment">
8	
9	<LinearLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:orientation="vertical"
13	android:paddingBottom="16dp">
14	
15	<TextView
16	android:layout_width="wrap_content"
17	android:layout_height="wrap_content"
18	android:layout_margin="16dp"
19	android:text="Featured Friends"
20	android:textSize="20sp"
21	android:textStyle="bold" />
22	
23	<androidx.recyclerview.widget.RecyclerView
24	android:id="@+id/rvFeaturedFriends"
25	android:layout_width="match_parent"
26	android:layout_height="wrap_content"
27	android:clipToPadding="false"
28	android:orientation="horizontal"

29	android:paddingHorizontal="8dp"
30	
	app:layoutManager="androidx.recyclerview.widget.LinearLa youtManager" />
31	
32	<TextView
33	android:layout_width="wrap_content"
34	android:layout_height="wrap_content"
35	android:layout_margin="16dp"
36	android:text="All Friends"
37	android:textSize="20sp"
38	android:textStyle="bold" />
39	
40	<androidx.recyclerview.widget.RecyclerView
41	android:id="@+id/rvAllFriends"
42	android:layout_width="match_parent"
43	android:layout_height="wrap_content"
44	android:nestedScrollingEnabled="false"
45	
	app:layoutManager="androidx.recyclerview.widget.LinearLa youtManager" />
46	
47	</LinearLayout>
48	</androidx.core.widget.NestedScrollView>

Item_friend.xml

Tabel 11 Source Code item_friend.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	
	xmlns:android="http://schemas.android.com/apk/res/andro id"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_marginHorizontal="16dp"
8	android:layout_marginVertical="8dp"
9	app:cardCornerRadius="16dp"
10	app:cardElevation="2dp">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="horizontal"
16	android:padding="16dp"

17	android:gravity="center_vertical">
18	
19	<com.google.android.material.imageview.ShapeableImageVi
20	ew
21	android:id="@+id/ivPhoto"
22	android:layout_width="100dp"
23	android:layout_height="100dp"
24	android:scaleType="centerCrop"
25	app:shapeAppearanceOverlay="@style/RoundedImage" />
26	<LinearLayout
27	android:layout_width="0dp"
28	android:layout_height="wrap_content"
29	android:layout_weight="1"
30	android:layout_marginStart="16dp"
31	android:orientation="vertical">
32	
33	<LinearLayout
34	android:layout_width="match_parent"
35	android:layout_height="wrap_content"
36	android:orientation="horizontal"
37	android:gravity="top">
38	
39	<TextView
40	android:id="@+id/tvName"
41	android:layout_width="0dp"
42	android:layout_height="wrap_content"
43	android:layout_weight="1"
44	android:text="Nama Teman"
45	android:textSize="16sp"
46	android:textStyle="bold"
47	android:maxLines="2"
48	android:ellipsize="end"/> <TextView
49	android:id="@+id/tvDate"
50	android:layout_width="wrap_content"
51	android:layout_height="wrap_content"
52	android:text="Tanggal"
53	android:textSize="10sp"
54	android:layout_marginStart="8dp"/>
55	</LinearLayout>
56	
57	<TextView
58	android:id="@+id/tvFeature"
59	android:layout_width="match_parent"

60	android:layout_height="wrap_content"
61	android:layout_marginTop="8dp"
62	android:text="Feature: Deskripsi
63	teman..."
64	android:textSize="12sp"
65	android:maxLines="2"
66	android:ellipsize="end"/>
67	<LinearLayout
68	android:layout_width="match_parent"
69	android:layout_height="wrap_content"
70	android:orientation="horizontal"
71	android:layout_marginTop="12dp">
72	
73	<Button
74	android:id="@+id/btnInstagram"
75	android:layout_width="0dp"
76	android:layout_height="wrap_content"
77	android:layout_weight="1"
78	android:text="Instagram"
79	android:textSize="11sp"
80	android:layout_marginEnd="4dp"/>
81	
82	<Button
83	android:id="@+id/btnDetail"
84	android:layout_width="0dp"
85	android:layout_height="wrap_content"
86	android:layout_weight="1"
87	android:text="Detail"
88	android:textSize="11sp"
89	android:layout_marginStart="4dp"/>
90	</LinearLayout>
91	
92	</LinearLayout>
93	</LinearLayout>
94	</com.google.android.material.card.MaterialCardView>

Nav_graph.xml

Tabel 12 Source Code nav_graph.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"

3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/friendListFragment">
7	
8	<fragment
9	android:id="@+id/friendListFragment"
10	android:name="com.example.modul3xml.ui.FriendListFragme
	nt"
11	android:label="Friend List"
12	tools:layout="@layout/fragment_friend_list">
13	
14	<action
15	android:id="@+id/action_friendListFragment_to_detailFra
	gment"
16	app:destination="@id/detailFragment" />
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	android:name="com.example.modul3xml.ui.DetailFragment"
22	android:label="Detail Friend"
23	tools:layout="@layout/fragment_detail">
24	
25	<argument
26	android:name="friendId"
27	app:argType="integer"
28	android:defaultValue="0" />
29	</argument>
30	
31	</navigation>

Source Code Compose

MainActivity.kt

Tabel 13. Source Code MainActivity.kt Jawaban Soal 1 Compose

1	package com.example.modul4compose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.foundation.layout.padding
9	import androidx.compose.material3.Scaffold
10	import androidx.compose.ui.Modifier
11	import
	com.example.modul4compose.theme.Modul3ComposeTheme
12	import com.example.modul4compose.uiapp.AppNavigation
13	
14	class MainActivity : ComponentActivity() {
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	enableEdgeToEdge()
18	setContent {
19	Modul3ComposeTheme {
20	Scaffold(modifier =
	Modifier.fillMaxSize()) { innerPadding ->
21	// hanya memanggil komponen
	navigasinya di sini
22	AppNavigation(
23	modifier =
	Modifier.padding(innerPadding)
24	)
25	}
26	}
27	}
28	}
29	}

FriendData

Tabel 14 Source Code Friend Data Jawaban Soal 1 Compose

a.	package com.example.modul4compose.data
b.	
c.	import com.example.modul4compose.R
d.	import com.example.modul4compose.model.Friend

e.	
f.	object FriendData {
g.	val friends = listOf(
h.	Friend(
i.	id = 1,
j.	name = "Muhammad Lv Gunawuy",
k.	date = "32 Agustus 6969",
l.	feature = "Foto lv ini diambil ketika dia ingin bercita cita menguasai dunia dan bertekad bahwa dia akan menjadi heavenly demon perkap yang tidak terkalahkan dan juga akan mmenentukan penerus heavenly demon perkap selanjutnya di bawah kepemimpinan dia",
m.	photoResId = R.drawable.lv,
n.	instagramUrl = "https://instagram.com/"
o.	),
p.	Friend(
q.	id = 2,
r.	name = "Fatih Bari Muar",
s.	date = "29 Masehi 3802",
t.	feature = "Fatih merupakan seorang puitisi yang sangat tidak terkenal dan tidak akan terkenal kerna puitisi dia hanya dapat digunakan dalam sekali pakai, foto ini diambil ketika dia sedang ingin buang hajat dan ingin mencari insiparsi untuk puisi yang akan dia buat selanjutnya",
u.	photoResId = R.drawable.fatih,
v.	instagramUrl = "https://instagram.com/"
w.	),
x.	Friend(
y.	id = 3,
z.	name = "Meluyyyyyyyy",
aa.	date = "32 March 12738",
bb.	feature = "Foto ini ketika Meluy sedang bermain sebua aplikasi seperti ome tv dan sedang mencari para cwk cwk yang akan digoda oleh meluy tetapi dia malah bertemu seorang pria dan dihina hina oleh pria tersebut sebagai chain smoker",
cc.	photoResId = R.drawable.melon,
dd.	instagramUrl = "https://instagram.com/"
ee.	),
ff.	Friend(
gg.	id = 4,
hh.	name = "neRvers.",
ii.	date = "43 April 21387",
jj.	feature = "Ini adalah foto seorang neRvers. ketika sedang lock in untuk melakukan ibadah, tapi disini

	dia sedang lock in karena setelah selesai ibadah dia ingin mencucuk pentol amang amang di muka masjid",
kk.	photoResId = R.drawable.nafis,
ll.	instagramUrl = "https://instagram.com/"
mm.	),
nn.	Friend(
oo.	id = 5,
pp.	name = "Rakha x Police",
qq.	date = "48 Janus 2317",
rr.	feature = "Foto ini adalah foto seorang yang ingin bercita cita menjadi seorang police yaitu rakha TI 24 FT ULM WAKAHIM HMTI FT ULM yang diman foto ini dia sedang melakukan wawancara kepada calon calon anggota HMTI FT ULM dengan dia berniat untuk menggagalkan semua orang",
ss.	photoResId = R.drawable.rakha,
tt.	instagramUrl = "https://instagram.com/"
uu.	)
vv.	)
ww.	}

Friend

Tabel 15 Source Code Friend Jawaban Soal 1 Compose

1	package com.example.modul3compose.model
2	
3	import androidx.annotation.DrawableRes
4	
5	data class Friend(
6	val id: Int,
7	val name: String,
8	val date: String,
9	val feature: String,
10	@DrawableRes val photoResId: Int,
11	val instagramUrl: String
12	)

DetailScreen.kt

Tabel 16 Source Code DetailScreen.kt Jawaban Soal 1 Compose

1	package com.example.modul4compose.uiapp
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.ui.Alignment
11	import androidx.compose.ui.Modifier
12	import androidx.compose.ui.draw.clip
13	import androidx.compose.ui.layout.ContentScale
14	import androidx.compose.ui.res.painterResource
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.unit.dp
17	import androidx.compose.ui.unit.sp
18	import com.example.modul4compose.data.FriendData
19	
20	@Composable
21	fun DetailScreen(friendId: Int, modifier: Modifier =
	Modifier) {
22	// Mencari data teman di FriendData berdasarkan ID
	yang dikirim
23	val friend = FriendData.friends.find { it.id ==
	friendId }
24	
25	if (friend != null) {
26	Column(
27	modifier = modifier
28	.fillMaxSize()
29	.padding(16.dp)
30	.verticalScroll(rememberScrollState())
31	) {
32	Image(
33	painter = painterResource(id =
	friend.photoResId),
34	contentDescription = friend.name,
35	contentScale = ContentScale.Crop,
36	modifier = Modifier
37	.fillMaxWidth()
38	.height(350.dp)
39	.clip(RoundedCornerShape(16.dp))

```

40         )
41
42         Spacer(modifier = Modifier.height(16.dp))
43
44         // Teks Judul
45         Text(
46             text = friend.name,
47             fontSize = 28.sp,
48             fontWeight = FontWeight.Bold,
49             lineHeight = 32.sp
50         )
51
52         //Tanggal / Keterangan Waktu
53         Text(
54             text = friend.date,
55             fontSize = 16.sp,
56             color =
MaterialTheme.colorScheme.onSurfaceVariant,
57             modifier = Modifier.padding(top = 4.dp,
bottom = 16.dp)
58         )
59
60         // Label Deskripsi
61         Text(
62             text = "Feature:",
63             fontSize = 18.sp,
64             fontWeight = FontWeight.Bold
65         )
66
67         Spacer(modifier = Modifier.height(4.dp))
68
69         // Isi Deskripsi Lengkap
70         Text(
71             text = friend.feature,
72             fontSize = 16.sp,
73             lineHeight = 24.sp
74         )
75     }
76     } else {
77         // if tidak ditemukan
78         Box(modifier = Modifier.fillMaxSize(),
contentAlignment = Alignment.Center) {
79             Text("Data teman tidak ditemukan")
80         }
81     }
82 }

```

FriendItem.kt

Tabel 17 Source Code FriendItem.kt Jawaban Soal 1 Compose

1	package com.example.modul4compose.uiapp
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
10	import androidx.compose.ui.layout.ContentScale
11	import androidx.compose.ui.platform.LocalContext
12	import androidx.compose.ui.res.painterResource
13	import androidx.compose.ui.text.font.FontWeight
14	import androidx.compose.ui.text.style.TextOverflow
15	import androidx.compose.ui.unit.dp
16	import androidx.compose.ui.unit.sp
17	import com.example.modul4compose.model.Friend
18	
19	@Composable
20	fun FriendItem(
21	friend: Friend,
22	onDetailClick: () -> Unit,
23	onInstagramClick: (String) -> Unit,
24	modifier: Modifier = Modifier
25	) {
26	val context = LocalContext.current
27	
28	Card(
29	shape = RoundedCornerShape(16.dp),
30	colors = CardDefaults.cardColors(containerColor
	= MaterialTheme.colorScheme.surfaceVariant),
31	modifier = modifier
32	.padding(horizontal = 8.dp, vertical = 8.dp)
33	) {
34	Row(
35	modifier = Modifier.padding(16.dp),
36	horizontalArrangement
	Arrangement.spacedBy(16.dp)
37	) {
38	Image(
39	painter = painterResource(id =
	friend.photoResId),
40	contentDescription = friend.name,

```

41         contentScale = ContentScale.Crop,
42         modifier = Modifier
43             .size(100.dp)
44             .clip(RoundedCornerShape(12.dp))
45     )
46
47     Column(
48         modifier = Modifier.weight(1f)
49     ) {
50
51         Row(
52             modifier = Modifier.fillMaxWidth(),
53             horizontalArrangement =
Arrangement.SpaceBetween
54         ) {
55
56             Text(
57                 text = friend.name,
58                 fontWeight = FontWeight.Bold,
59                 fontSize = 16.sp,
60                 maxLines = 1,
61                 overflow =
TextOverflow.Ellipsis,
62                 modifier = Modifier.weight(1f)
63             )
64
65             Text(
66                 text = friend.date,
67                 fontSize = 10.sp,
68                 modifier =
Modifier.padding(start = 8.dp)
69             )
70
71         }
72
73         Spacer(modifier =
Modifier.height(8.dp))
74
75         Text(
76             text = friend.feature,
77             fontSize = 12.sp,
78             maxLines = 4,
79             overflow = TextOverflow.Ellipsis
80         )
81
82         Spacer(modifier =
Modifier.height(12.dp))

```

```

83
84         Row(
85             modifier = Modifier.fillMaxWidth(),
86             horizontalArrangement =
Arrangement.spacedBy(8.dp)
87         ) {
88
89             Button(
90                 onClick = {
onInstagramClick(friend.instagramUrl) }, // Panggil
fungsi di sini
91                 contentPadding =
PaddingValues(horizontal = 4.dp, vertical = 8.dp),
92                 modifier = Modifier.weight(1f),
93                 colors =
ButtonDefaults.buttonColors()
94             ) {
95                 Text("Instagram", fontSize =
11.sp, maxLines = 1, overflow = TextOverflow.Ellipsis)
96             }
97
98             Button(
99                 onClick = onDetailClick,
100                 contentPadding =
PaddingValues(horizontal = 4.dp, vertical = 8.dp),
101                 modifier = Modifier.weight(1f),
102                 colors =
ButtonDefaults.buttonColors()
103             ) {
104                 Text("Detail", fontSize = 11.sp,
maxLines = 1, overflow = TextOverflow.Ellipsis)
105             }
106         }
107     }
108 }
109 }
110 }

```

FriendListScreen.kt

Tabel 18 Source Code FriendListScreen.kt Jawaban Soal 1 Compose

```
1 package com.example.modul4compose.uiapp
2
3 import androidx.compose.foundation.layout.*
4 import androidx.compose.foundation.lazy.LazyColumn
5 import androidx.compose.foundation.lazy.LazyRow
6 import androidx.compose.foundation.lazy.items
7 import androidx.compose.material3.Text
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.text.font.FontWeight
11 import androidx.compose.ui.unit.dp
12 import androidx.compose.ui.unit.sp
13 import com.example.modul4compose.model.Friend
14
15 @Composable
16 fun FriendListScreen(
17     friends: List<Friend>,
18     onNavigateToDetail: (Friend) -> Unit,
19     onInstagramClick: (String) -> Unit,
20     modifier: Modifier = Modifier
21 ) {
22     LazyColumn(
23         modifier = modifier.fillMaxSize(),
24     ) {
25         item {
26             Text(
27                 text = "Featured Friends",
28                 fontWeight = FontWeight.Bold,
29                 fontSize = 20.sp,
30                 modifier = Modifier.padding(start = 16.dp, top = 16.dp, bottom = 8.dp)
31             )
32         }
33
34         item {
35             LazyRow(
36                 contentPadding =
37                 PaddingValues(horizontal = 16.dp),
38                 horizontalArrangement =
39                 Arrangement.spacedBy(16.dp)
40             ) {
41                 items(friends.take(5)) { friend ->
42                     Box(modifier =
43                     Modifier.width(350.dp)) {
```

```

41         FriendItem(
42             friend = friend,
43             onDetailClick = {
onNavigateToDetail(friend) },
44             onInstagramClick = {
onInstagramClick(friend.instagramUrl) }
45         )
46     }
47 }
48 }
49 }
50
51     item {
52         Text(
53             text = "All Friends",
54             fontWeight = FontWeight.Bold,
55             fontSize = 20.sp,
56             modifier = Modifier.padding(start =
16.dp, top = 24.dp, bottom = 8.dp)
57         )
58     }
59
60     items(friends) { friend ->
61         FriendItem(
62             friend = friend,
63             onDetailClick = {
onNavigateToDetail(friend) },
64             onInstagramClick = {
onInstagramClick(friend.instagramUrl) }
65         )
66     }
67 }
68 }

```

AppNavigation

Tabel 19 Source Code AppNavigation.kt Jawaban Soal 1 Compose

```

1 package com.example.modul4compose.uiapp
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.runtime.Composable
6 import androidx.compose.runtime.collectAsState
7 import androidx.compose.runtime.getValue
8 import androidx.compose.ui.Modifier
9 import androidx.compose.ui.platform.LocalContext

```

```

10 import androidx.lifecycle.viewmodel.compose.viewModel
11 import androidx.navigation.NavType
12 import androidx.navigation.compose.NavHost
13 import androidx.navigation.compose.composable
14 import androidx.navigation.compose.rememberNavController
15 import androidx.navigation.navArgument
16 import timber.log.Timber
17
18 @Composable
19 fun AppNavigation(modifier: Modifier = Modifier) {
20     val navController = rememberNavController()
21
22     // Inisialisasi ViewModel dengan Factory
23     val viewModel: FriendViewModel = viewModel(
24         factory = FriendViewModelFactory("Fathi")
25     )
26
27     NavHost(
28         navController = navController,
29         startDestination = "list_screen",
30         modifier = modifier
31     ) {
32         // HALAMAN 1: LIST SCREEN
33         composable("list_screen") {
34             val friends by
viewModel.friendsState.collectAsState()
35             val context = LocalContext.current
36
37             FriendListScreen(
38                 friends = friends,
39                 onNavigateToDetail = { friend ->
40                     // Log saat tombol Detail ditekan
41                     // Log data dari list yang dipilih
42                     viewModel.onFriendClicked(friend)
43
44                     navController.navigate("detail_screen/${friend.id}")
45                     },
46                     onInstagramClick = { url ->
47                         // Log saat tombol Explicit Intent
48                         (Instagram) ditekan
49                         Timber.d("Tombol Explicit Intent
50                         (Instagram) ditekan. Membuka URL: $url")
51
52                         // Logika Explicit Intent
53                         val intent =
54                         Intent(Intent.ACTION_VIEW, Uri.parse(url))
55                         context.startActivity(intent)

```

```

52         }
53     )
54 }
55
56 // HALAMAN 2: DETAIL SCREEN (Dikembalikan lagi
ke sini)
57     composable(
58         route = "detail_screen/{friendId}",
59         arguments = listOf(navArgument("friendId") {
type = NavType.IntType })
60     ) { backStackEntry ->
61         val id =
backStackEntry.arguments?.getInt("friendId") ?: 0
62         DetailScreen(friendId = id)
63     }
64 }
65 }

```

FriendViewModel

Tabel 20 Source Code FriendViewModel.kt Jawaban Soal 1 Compose

```

1 package com.example.modul4compose.uiapp
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import com.example.modul4compose.data.FriendData
6 import com.example.modul4compose.model.Friend
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.asStateFlow
10 import timber.log.Timber
11
12 class FriendViewModel(private val userName: String) :
ViewModel() {
13
14     // StateFlow untuk daftar kawan
15     private val _friendsState =
MutableStateFlow<List<Friend>>(emptyList())
16     val friendsState: StateFlow<List<Friend>> =
_friendsState.asStateFlow()
17
18     init {
19         loadFriends()
20         Timber.d("ViewModel diinisialisasi oleh:
$username")
21     }

```

```

22
23     private fun loadFriends() {
24         val data = FriendData.friends
25         _friendsState.value = data
26         Timber.d("Data item masuk ke dalam list:
27     ${data.size} friend")
28     }
29
30     // Fungsi untuk log ketika button ditekan
31     fun onFriendClicked(friend: Friend) {
32         Timber.d("Button Detail ditekan untuk:
33     ${friend.name}")
34         Timber.d("Berpindah ke halaman Detail dengan
35     data: $friend")
36     }
37
38     // ViewModel Factory parameter String
39     class FriendViewModelFactory(private val name: String) :
40     ViewModelProvider.Factory {
41         override fun <T : ViewModel> create(modelClass:
42     Class<T>): T {
43             if
44             (modelClass.isAssignableFrom(FriendViewModel::class.java)) {
45                 @Suppress("UNCHECKED_CAST")
46                 return FriendViewModel(name) as T
47             }
48             throw IllegalArgumentException("Unknown
49     ViewModel class")
50         }
51     }

```

MainApplication

Tabel 21 Source Code MainApplication.kt Jawaban Soal 1 Compose

```

1 package com.example.modul4compose
2
3 import android.app.Application
4 import timber.log.Timber
5
6 class MainApplication : Application() {
7     override fun onCreate() {
8         super.onCreate()
9         // Inisialisasi Timber hanya pada mod debug agar
10        log tidak muncul di production

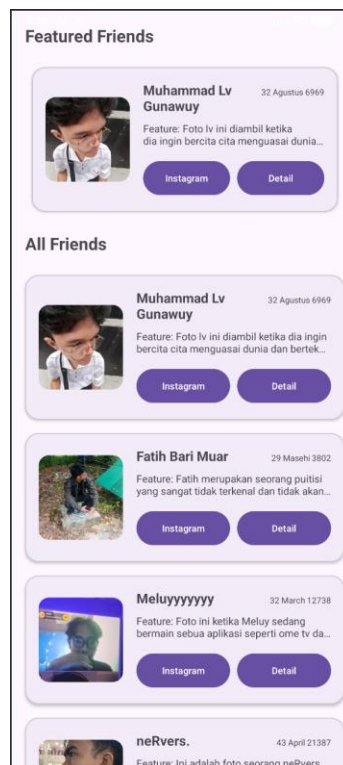
```

```

10         val isDebuggable = (applicationInfo.flags and
11         android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE) != 0
12         if (isDebuggable) {
13             Timber.plant(Timber.DebugTree())
14         }
15     }

```

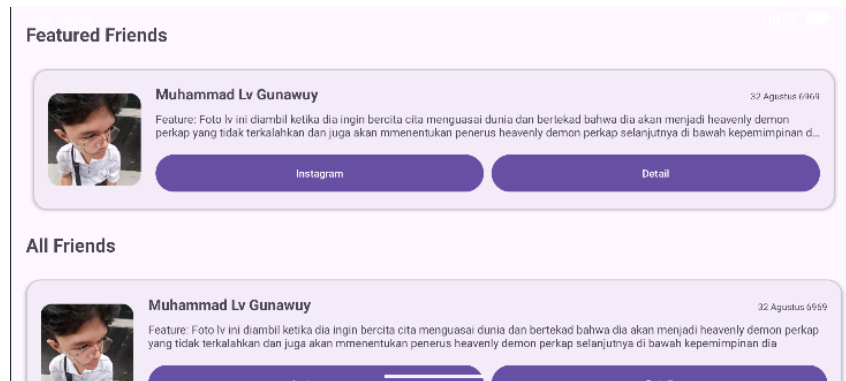
B. Output Program



Gambar 2. Tampilan Awal Aplikasi XML (Vertikal)



Gambar 3. Tampilan Halaman Detail Aplikasi XML (Vertikal)



Gambar 4 Tampilan Aplikasi XML (Horizontal)



Gambar 5 Tampilan Halaman Detail Aplikasi XML (Horizontal)

2026-05-15 17:15:01.833	1224-3365	WM-NetworkStateTracker	com.google.android.ext.services	D Network capabilities changed: [Transports: WIFI Capabili
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D Data item masuk ke dalam list: 5 kawan
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D ViewModel diinisialisasi oleh: Fathi

Gambar 6 Hasil Logcat dari XML stasaat Masuk Aplikasi

2026-05-15 17:15:01.833	1224-3365	WM-NetworkStateTracker	com.google.android.ext.services	D Network capabilities changed: [Transports: WIFI Capabili
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D Data item masuk ke dalam list: 5 kawan
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D ViewModel diinisialisasi oleh: Fathi
2026-05-15 17:28:12.563	4393-4393	FriendAdap...ViewHolder	com.example.modul3xml	D Tombol Explicit Intent (Instagram) ditekan. Membuka URL:
2026-05-15 17:28:16.378	3383-3383	cr_OfflineIndicator	com.android.chrome	I Reported by system: [Transports: WIFI Capabilities: NOT_

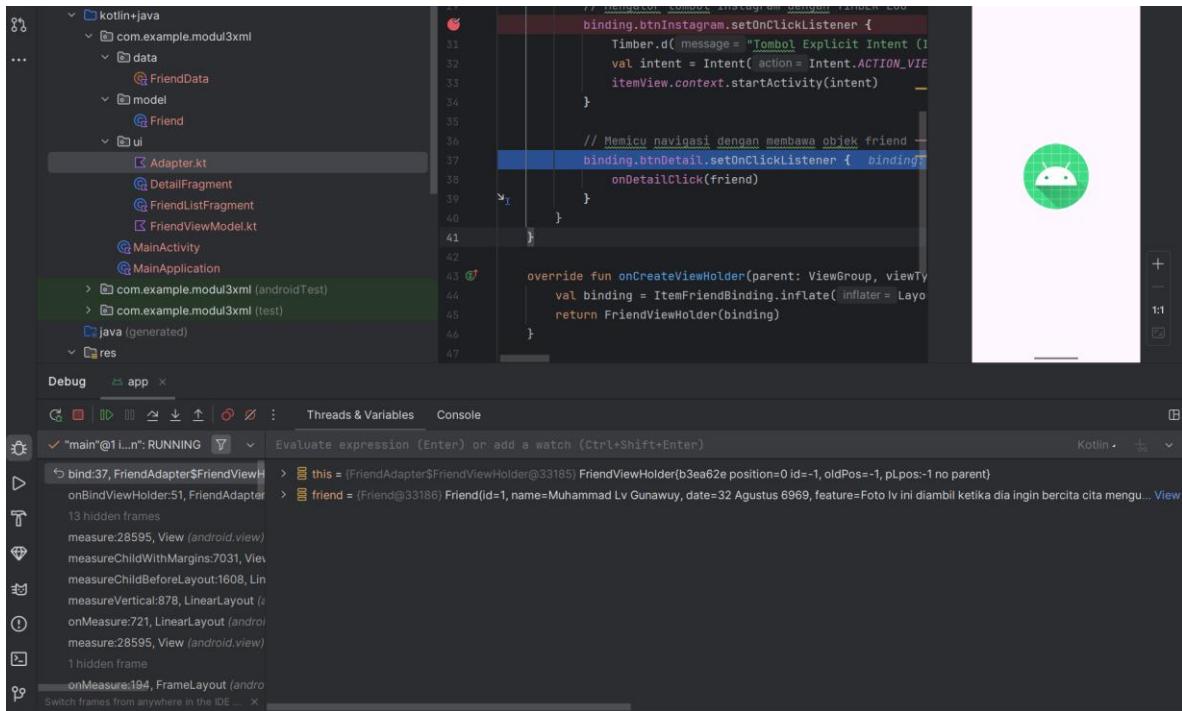
Gambar 7 Hasil Logcat dari XML saat Klik Instagram

2026-05-15 17:15:01.833	1224-3365	WM-NetworkStateTracker	com.google.android.ext.services	D Network capabilities changed: [Transports: WIFI Capabili
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D Data item masuk ke dalam list: 5 kawan
2026-05-15 17:20:12.141	4393-4393	FriendViewModel	com.example.modul3xml	D ViewModel diinisialisasi oleh: Fathi
2026-05-15 17:28:12.563	4393-4393	FriendAdap...ViewHolder	com.example.modul3xml	D Tombol Explicit Intent (Instagram) ditekan. Membuka URL:
2026-05-15 17:28:16.378	3383-3383	cr_OfflineIndicator	com.android.chrome	I Reported by system: [Transports: WIFI Capabilities: NOT_
2026-05-15 17:29:30.921	4393-4393	FriendViewModel	com.example.modul3xml	D Button Detail ditekan untuk: Muhammad Lv Sunawuy
2026-05-15 17:29:30.921	4393-4393	FriendViewModel	com.example.modul3xml	D Berpindah ke halaman Detail dengan data: Friend(id=1, na

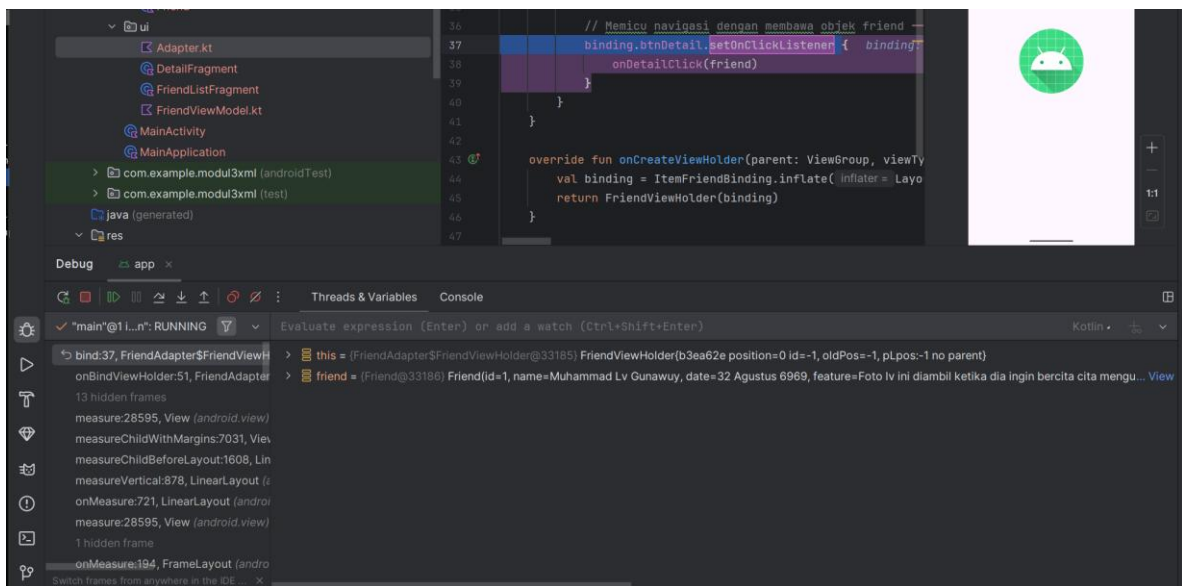
Gambar 8 Hasil Logcat dari XML saat Klik Detail

The screenshot displays the Android Studio environment. On the left, the 'Project' tab shows the file structure with 'com.example.modul3xml' selected. The central 'Text' tab shows the XML code for the 'friend' list item, featuring a binding for 'btnInstagram' and 'btnDetail'. The rightmost 'Preview' tab shows a visual representation of the app's UI. At the bottom, the 'Debug Console' is active, showing the state of the application, including the binding and the friend object.

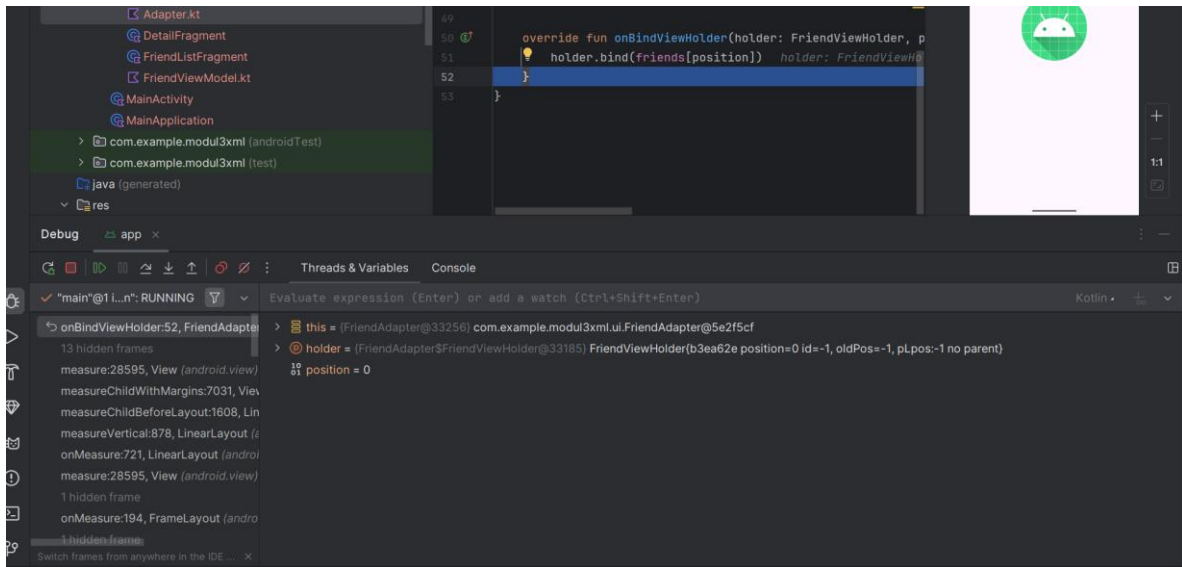
Gambar 9 Break Point dan Hasil Debug XML



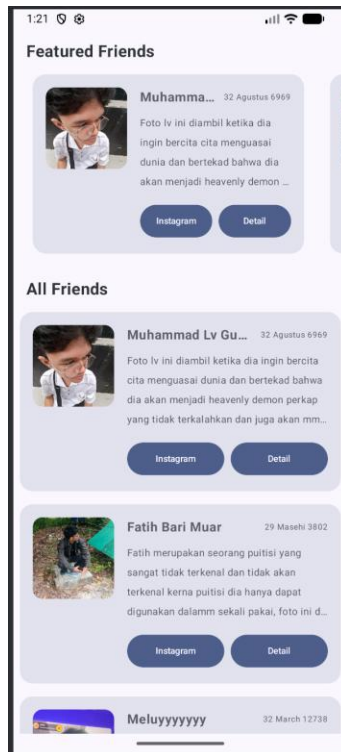
Gambar 10 Hasil Step Over XML



Gambar 11 Hasil Step Into XML



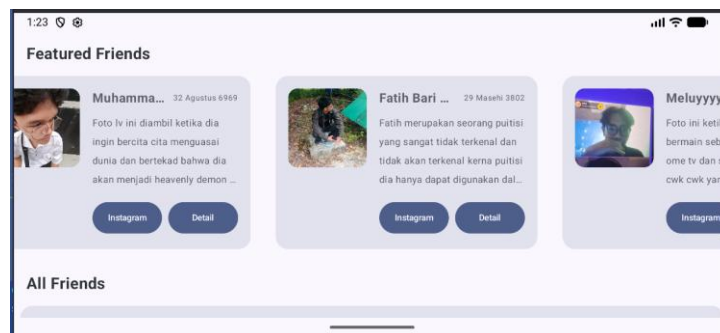
Gambar 12 Hasil Step Out XML



Gambar 13. Tampilan Awal Aplikasi Compose (Vertikal)



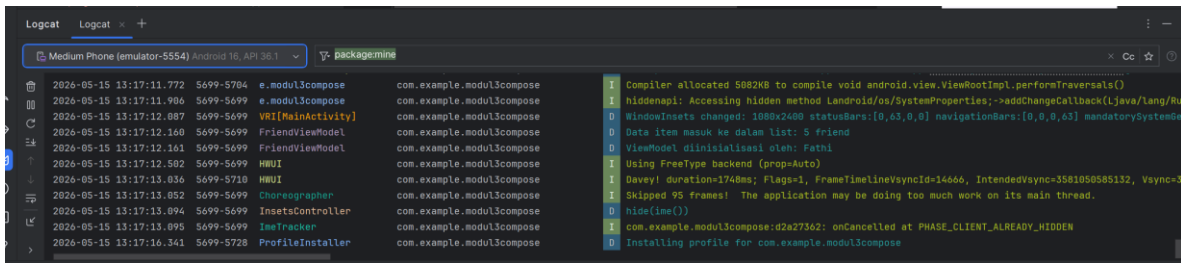
Gambar 14. Tampilan Halaman Detail Aplikasi Compose (Vertikal)



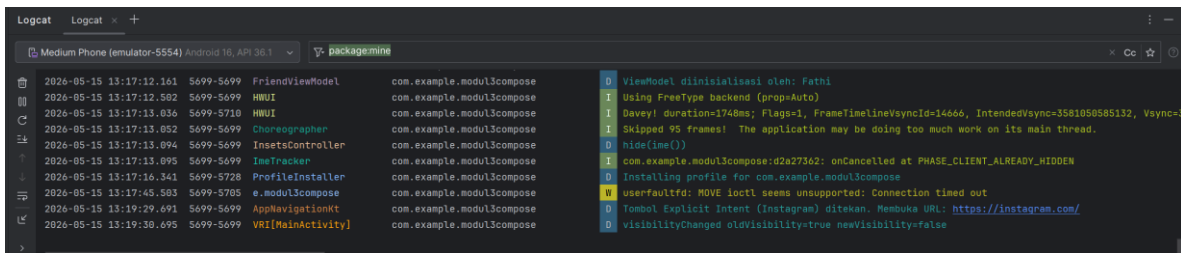
Gambar 15. Tampilan Aplikasi Compose (Horizontal)



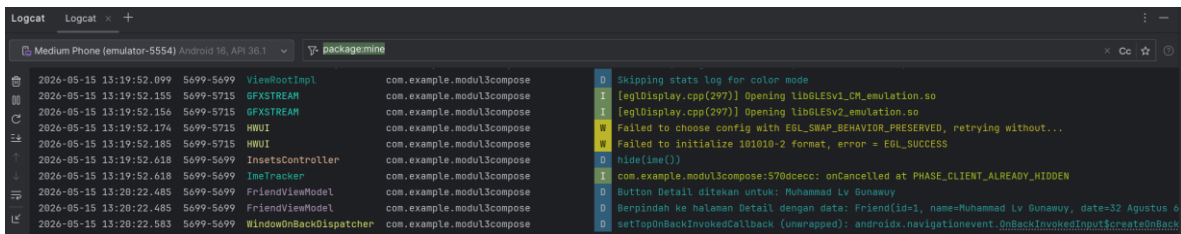
Gambar 16. Tampilan Halaman Detail Aplikasi Compose (Horizontal)



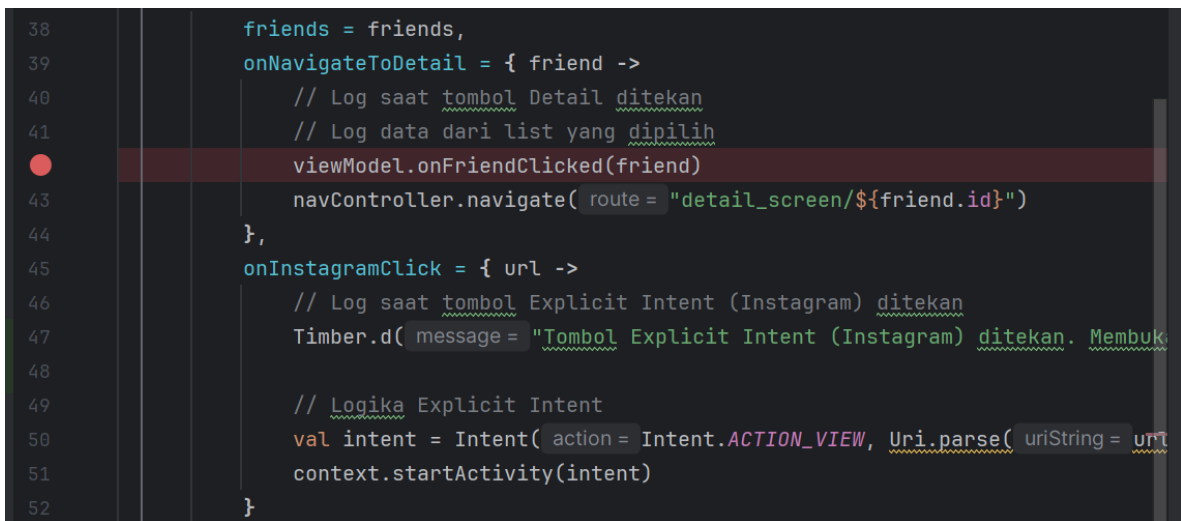
Gambar 17 Hasil Logcat dari Compose saat Masuk Aplikasi



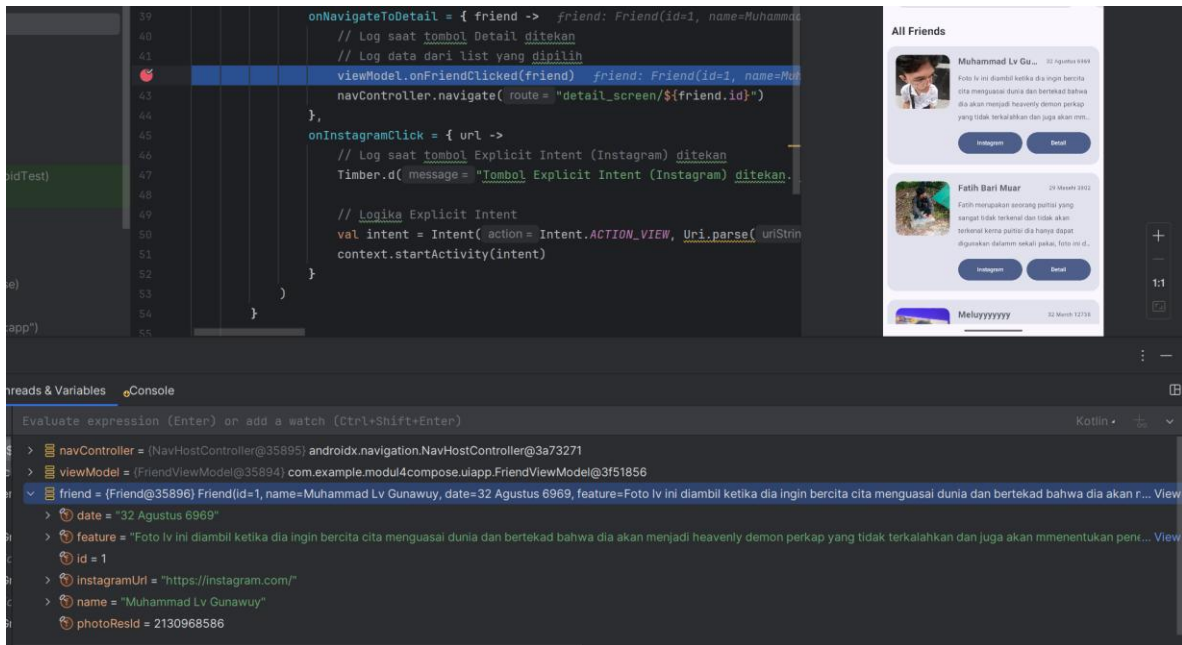
Gambar 18 Hasil Logcat dari Compose saat Klik Instagram



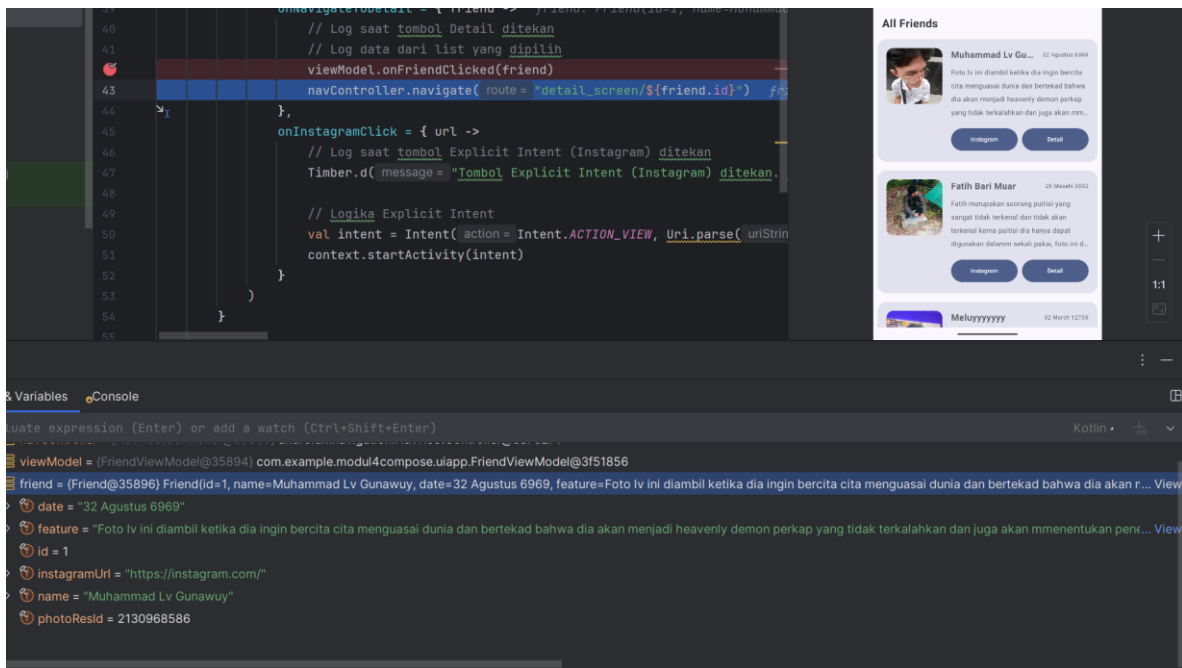
Gambar 19 Hasil Logcat dari Compose saat Klik Detail



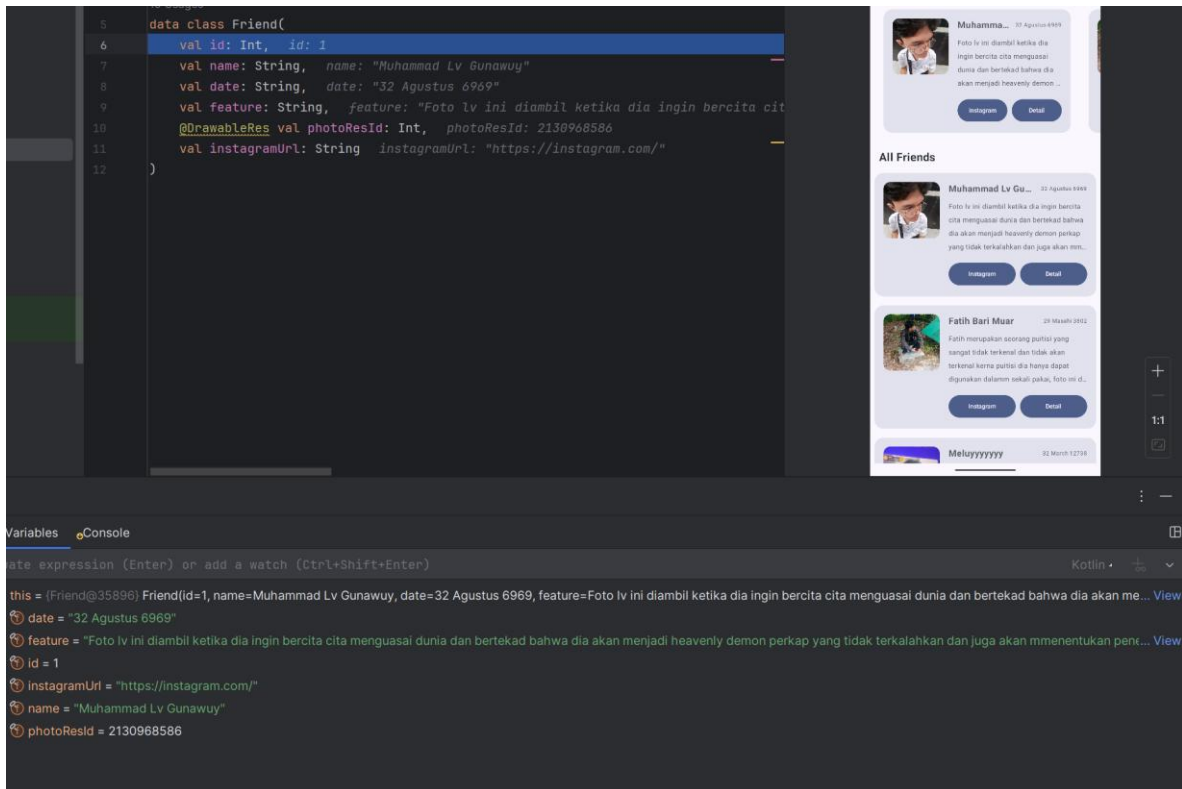
Gambar 20 Break Point Compose



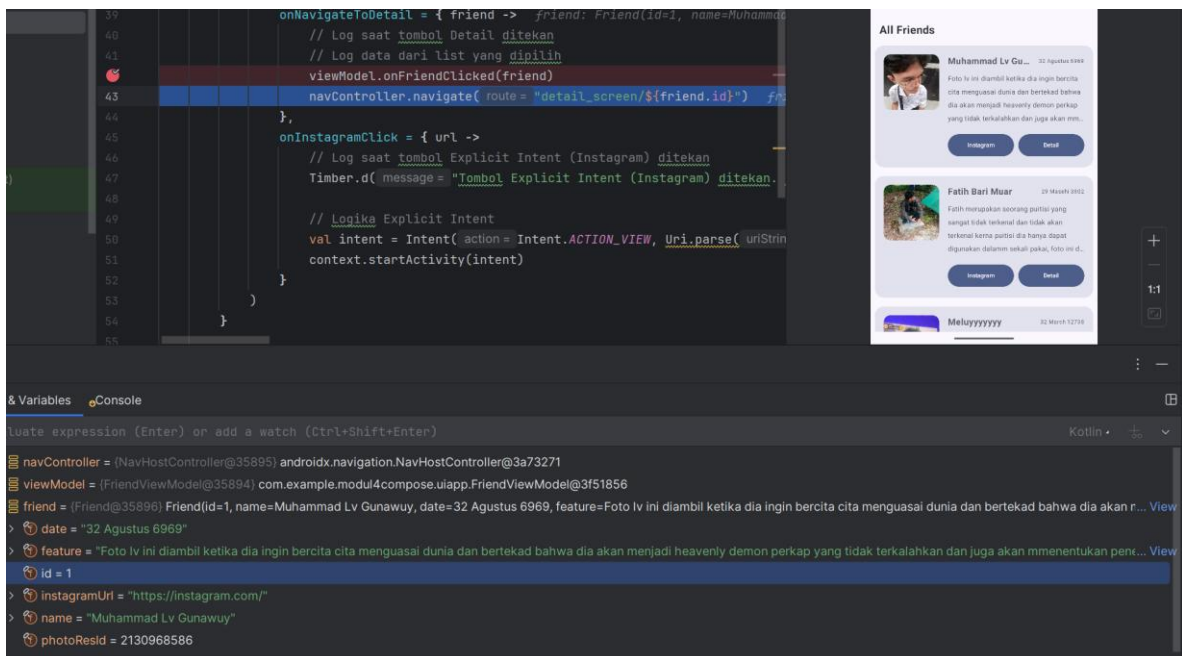
Gambar 21 Hasil Debug Compose



Gambar 22 Hasil Step Over Compose



Gambar 23 Hasil Step Into Compose



Gambar 24 Hasil Step Out Compose

C. Pembahasan

Pembahasan XML

FriendData:

Line 1: Deklarasi package `com.example.modul3xml.data`. Ini nandain kalau posisi file ini disimpan khusus di dalam folder data.

Line 3: Blok import (yang posisinya sedang dilipat/di-collapse di Android Studio-mu). Kalau dibuka, isinya memanggil `R` (buat ngambil file gambar dari dalam folder `res/drawable`) dan memanggil cetakan model `Friend` yang sebelumnya udah kamu buat.

Line 6: Deklarasi object `FriendData {`. Penggunaan kata object di Kotlin (bukan class) berarti kita membuat Singleton. Intinya, gudang data ini cuma diciptakan satu kali di memori HP, jadi lebih ringan dan bisa dipanggil kapan aja dari halaman manapun tanpa harus dibikin ulang.

Line 7: `val friends = listOf(`. Kita bikin variabel bernama `friends` yang tipenya adalah `List` (daftar berurutan). Semua data temanmu bakal dibungkus dan diurutkan di dalam sini.

Line 8: Data teman pertama. Kita manggil cetakan `Friend()` lalu ngisi datanya sesuai urutan: ID (1), Nama ("`Muhammad Lv Gunawuy`"), Tanggal nyelenehnya ("`32 Agustus 6969`"), cerita panjang soal heavenly demon, ID buat manggil fotonya (`R.drawable.lv`), dan link Instagram.

Line 9: Data teman kedua. Mengisi ID 2 untuk "`Fatih Bari Muar`" lengkap dengan tanggal "`29 Masehi 3802`", cerita soal inspirasi puisi di kamar mandi, dan ID fotonya (`R.drawable.fatih`).

Line 10: Data teman ketiga. Mengisi ID 3 untuk "`Meluyyyyyyyy`", cerita lucu soal aplikasi video chat, dan ID fotonya (`R.drawable.melon`).

Line 11: Data teman keempat. Mengisi ID 4 untuk "`neRvers.`", cerita soal lock in ibadah dan pentol di muka masjid, serta ID fotonya (`R.drawable.nafis`).

Line 12: Data teman kelima. Mengisi ID 5 untuk "`Rakha x Police`", cerita soal wawancara calon anggota HMTI FT ULM, dan ID fotonya (`R.drawable.rakha`).

Line 13: Kurung tutup) untuk mengakhiri batas isi dari listOf yang tadi dibuka di baris 7.

Line 14: Kurung kurawal tutup } untuk mengakhiri blok dari object FriendData.

Friend:

Line 1: Deklarasi package com.example.modul3xml.model. Ini menunjukkan kalau file ini disimpan di dalam folder model. Dalam dunia koding, folder model memang tempat khusus untuk menaruh rancangan atau cetakan dasar struktur data.

Line 3: Mengimpor androidx.annotation.DrawableRes. Ini adalah alat bantu penanda (anotasi) dari Android yang nanti dipakai di baris 10 untuk memastikan kevalidan file gambar.

Line 5: data class Friend(. Mendeklarasikan sebuah data class bernama Friend. Di Kotlin, data class itu ibarat formulir kosong yang memang diciptakan khusus untuk menyimpan data saja, tanpa ada rumus atau logika rumit di dalamnya. Tanda kurung buka (berarti kita mulai mendaftar isian formulirnya.

Line 6: val id: Int,. Isian pertama adalah id bertipe Int (angka bilangan bulat). Ini fungsinya seperti nomor antrean atau nomor identitas unik agar sistem gampang mencari data spesifik temanmu.

Line 7: val name: String,. Isian kedua, name bertipe String (teks). Digunakan untuk menyimpan nama teman.

Line 8: val date: String,. Isian ketiga, date bertipe String. Digunakan untuk menyimpan keterangan waktu atau tanggal nyeleneh dari setiap foto.

Line 9: val feature: String,. Isian keempat, feature bertipe String. Ini wadah untuk menampung teks cerita atau deskripsi panjang temanmu.

Line 10: @DrawableRes val photoResId: Int,. Isian kelima, photoResId bertipe Int. Nah, tambahan @DrawableRes di depannya adalah hasil dari import di baris 3 tadi. Fungsinya sebagai "satpam" yang memastikan kalau angka yang dimasukkan ke sini wajib berupa ID gambar resmi dari dalam folder Android (seperti R.drawable.fatih), bukan angka sembarangan.

Line 11: `val instagramUrl: String`. Isian keenam atau yang terakhir, `instagramUrl` bertipe `String`. Ini dipakai untuk menyimpan alamat link Instagram yang nanti akan disambungkan ke tombol agar bisa pindah aplikasi.

Line 12: `)`. Kurung tutup untuk mengakhiri daftar isian dari cetakan data class `Friend` tersebut.

Adapter.kt

Line 1: Deklarasi package `com.example.modul3xml.ui` yang menentukan ruang lingkup dan lokasi hierarki file adapter ini di dalam direktori antarmuka pengguna proyek.

Line 3 - 11: Kumpulan perintah `import` untuk menyertakan kelas dan pustaka yang diperlukan:

- `android.content.Intent` dan `android.net.Uri`: Kelas untuk mengeksekusi perpindahan aplikasi (Explicit Intent) dan memproses tautan URL.
- `android.view.LayoutInflater` dan `ViewGroup`: Kelas pengelola tata letak untuk mengubah rancangan XML menjadi objek tampilan nyata di layar.
- `androidx.recyclerview.widget.RecyclerView`: Kelas inti pembuat daftar gulir dinamis.
- `com.bumptech.glide.Glide`: Pustaka pihak ketiga spesialis pemroses dan pemuat gambar secara efisien.
- `com.example.modul3xml.databinding.ItemFriendBinding`: Kelas hasil generate otomatis dari `ViewBinding` untuk mengakses elemen-elemen di dalam file XML `item_friend.xml` secara aman.
- `com.example.modul3xml.model.Friend` dan `timber.log.Timber`: Kelas model data kawan dan pustaka pencatat log.

Line 13 - 16: Mendeklarasikan kelas `FriendAdapter` yang mewarisi fungsi dari `RecyclerView.Adapter<FriendAdapter.FriendViewHolder>()`. Konstruktor utama kelas ini mewajibkan dua parameter:

- `friends: List<Friend>`: Kumpulan data teman yang akan dirender.

- `onDetailClick: (Friend) -> Unit`: Fungsi callback yang akan dijalankan ketika pengguna menekan tombol detail, dengan membawa serta objek kawan terkait.

Line 18: Mendeklarasikan inner class `FriendViewHolder` yang bertindak sebagai pola penahanan tampilan (view holder) dengan mewarisi `RecyclerView.ViewHolder`. Kelas ini menerima parameter binding berjenis `ItemFriendBinding` dan meneruskan akar tampilannya (`binding.root`) ke konstruktor superkelasnya.

Line 20: Membuka fungsi `bind(friend: Friend)` di dalam `ViewHolder`. Fungsi ini bertanggung jawab untuk memasukkan data dari objek friend tunggal ke dalam komponen-komponen antarmuka visual.

Line 21 - 23: Mengikat data teks ke dalam elemen antarmuka yang ada di XML melalui `ViewBinding`:

- Menyisipkan `friend.name` ke dalam `TextView` bernama `tvName`.
- Menyisipkan `friend.date` ke dalam `TextView` bernama `tvDate`.
- Menyisipkan gabungan string statis "Feature: " dan `friend.feature` ke dalam `TextView` bernama `tvFeature`.

Line 25 - 27: Memanggil pustaka `Glide` untuk memuat gambar. Proses ini mengambil konteks dari item view saat ini, memuat sumber daya gambar berdasarkan parameter `friend.photoResId`, dan menempelkan hasilnya ke dalam komponen `ImageView` bernama `ivPhoto`.

Line 30: Menetapkan pendengar kejadian klik (`setOnClickListener`) pada tombol Instagram (`binding.btnInstagram`).

Line 31: Mencetak log diagnostik menggunakan `Timber.d()` untuk merekam kejadian penekanan tombol beserta tautan URL yang akan dibuka.

Line 32: Mendirikan instruksi `Intent` dengan aksi `ACTION_VIEW` dan mengonversi format string `friend.instagramUrl` menjadi objek `Uri` yang dapat dibaca oleh sistem Android.

Line 33: Mengeksekusi peluncuran aktivitas untuk membuka peramban atau aplikasi eksternal berdasarkan instruksi dari variabel `intent` yang telah dibuat.

Line 34: Tanda kurung kurawal tutup } untuk mengakhiri instruksi pendengar klik pada tombol Instagram.

Line 37: Menetapkan pendengar kejadian klik (setOnClickListener) pada tombol Detail (binding.btnDetail).

Line 38: Mengeksekusi fungsi callback onDetailClick yang didefinisikan di konstruktor adapter, sembari melemparkan keseluruhan data objek friend saat ini agar dapat diproses lebih lanjut oleh Fragment atau Activity induk.

Line 39: Tanda kurung kurawal tutup } untuk mengakhiri instruksi pendengar klik pada tombol Detail.

Line 40: Tanda kurung kurawal tutup } untuk menyatakan selesainya fungsionalitas pengikatan data pada metode bind().

Line 41: Tanda kurung kurawal tutup } yang mendefinisikan batas akhir dari ruang lingkup inner class FriendViewHolder.

Line 43: Melakukan penimpaan (override) terhadap metode wajib onCreateViewHolder(). Sistem memanggil fungsi ini saat RecyclerView membutuhkan penahan tampilan baru akibat belum adanya tampilan yang dapat didaur ulang.

Line 44: Melakukan proses inflate (pengembangan) rancangan XML menjadi objek Java/Kotlin. Proses ini menggunakan LayoutInflater dari konteks parent (induk) dan dimasukkan ke dalam variabel binding.

Line 45: Mengembalikan instansiasi class FriendViewHolder yang baru dibentuk, dengan menyuntikkan variabel binding ke dalamnya.

Line 46: Tanda kurung kurawal tutup } untuk menyelesaikan implementasi fungsi onCreateViewHolder().

Line 48: Melakukan penimpaan fungsi getItemCount() dengan mengatur nilai balikkannya secara singkat menjadi ekuivalen dengan jumlah total elemen yang ada di dalam koleksi friends (friends.size). Angka ini memberitahu RecyclerView batasan proses rendering daftarnya.

Line 50: Melakukan penimpaan fungsi `onBindViewHolder()`. Fungsi pengikat ini dipanggil secara berulang oleh sistem `RecyclerView` untuk menampilkan data pada posisi tertentu.

Line 51: Menginstruksikan objek penahan tampilan (holder) untuk mengeksekusi metode `.bind()` miliknya, dengan memasukkan objek kawan tunggal yang diekstraksi dari dalam koleksi `friends` berdasarkan indeks lokasi yang sedang diproses (`friends[position]`).

Line 52: Tanda kurung kurawal tutup `}` yang menandakan berakhirnya siklus modifikasi pada metode `onBindViewHolder()`.

Line 53: Tanda kurung kurawal tutup `}` terakhir sebagai segel struktural yang membatasi seluruh ruang lingkup eksekusi pada kelas `FriendAdapter`.

DetailFragment.kt

Line 1: Deklarasi package `com.example.modul3xml.ui` yang mengelompokkan file ini ke dalam direktori komponen antarmuka pengguna pada hierarki proyek.

Line 3 - 10: Kumpulan instruksi import untuk memanggil kelas dan pustaka yang esensial, meliputi:

- `android.os.Bundle`: Kelas pengemas data untuk mentransfer informasi antar komponen navigasi.
- `android.view.*`: Mengimpor `LayoutInflater`, `View`, dan `ViewGroup` yang berfungsi untuk memproses dan menampung tata letak visual layar.
- `androidx.fragment.app.Fragment`: Kelas pondasi untuk membangun komponen `Fragment` yang memiliki siklus hidupnya sendiri.
- `com.bumptech.glide.Glide`: Pustaka eksternal terkemuka untuk pemrosesan dan asinkronisasi pemuatan gambar secara optimal.
- `com.example.modul3xml.data.FriendData`: Pangkalan data statis yang menyediakan daftar koleksi kawan.
- `com.example.modul3xml.databinding.FragmentDetailBinding`: Kelas jembatan yang dihasilkan otomatis oleh fitur `ViewBinding` untuk menghubungkan kode Kotlin dengan elemen-elemen di layout `fragment_detail.xml`.

Line 12: Mendeklarasikan kelas DetailFragment yang secara absolut mewarisi seluruh sifat dan siklus hidup dari superkelas Fragment().

Line 14: Mendeklarasikan variabel privat `_binding` dengan tipe data `FragmentDetailBinding?` (nullable). Variabel ini diinisialisasi dengan nilai null dan bertugas menyimpan referensi hierarki tampilan secara internal.

Line 15: Mendeklarasikan properti publik `binding` yang sifatnya read-only (hanya baca). Variabel ini menggunakan custom getter untuk mengambil nilai dari `_binding!!`. Tanda `!!` merupakan non-null assertion yang menginstruksikan kompilator bahwa variabel ini dijamin tidak akan bernilai null saat diakses. Praktik ini mempermudah pemanggilan ID komponen UI tanpa perlu melakukan pengecekan null terus-menerus.

Line 17: Melakukan penimpaan (override) pada metode siklus hidup `onCreateView()`. Fase ini dipanggil oleh sistem saat Fragment diminta untuk membentuk atau memuat tampilan antarmukanya (UI) untuk pertama kali.

Line 21: Mengeksekusi proses pengembangan (inflate) rancangan XML menjadi objek Java/Kotlin. Pemanggilan `FragmentDetailBinding.inflate` memanfaatkan inflater dan disematkan pada container (wadah induk), dengan bendera `false` agar tampilan tidak langsung ditempelkan secara paksa ke induknya sebelum proses perenderan selesai. Hasil objek tampilan ini disimpan ke dalam `_binding`.

Line 22: Mengembalikan akar dari tata letak visual (`binding.root`) agar sistem Android dapat merendernya ke layar perangkat.

Line 23: Tanda kurung kurawal tutup `}` yang mendefinisikan akhir dari instruksi pada fungsi `onCreateView()`.

Line 25: Melakukan penimpaan pada metode `onViewCreated()`. Fase ini otomatis dieksekusi tepat setelah `onCreateView()` berhasil mengembalikan tampilan. Ini adalah lokasi yang paling aman dan direkomendasikan untuk melakukan inisialisasi data serta manipulasi elemen UI.

Line 26: Memanggil fungsi `super.onViewCreated()` untuk mendelegasikan perintah ke kelas induk guna memastikan prosedur setup dasar bawaan sistem Android tetap tereksekusi.

Line 29: Mendeklarasikan variabel `friendId` untuk mengekstrak data ID yang dikirimkan melalui navigasi Bundle. Pemanggilan `arguments?.getInt("friendId")` menggunakan instrumen safe call (`?`), yang kemudian dipadukan dengan operator Elvis (`?: 0`). Jika kunci pengenalan `"friendId"` gagal ditemukan atau gagal diurai, sistem akan memberikan nilai cadangan (fallback) berupa angka 0.

Line 32: Mendeklarasikan variabel `friend` yang mencari objek spesifik dari dalam senarai `FriendData.friends`. Fungsi lambda `.find { it.id == friendId }` akan menelusuri data dan mencocokkan ID dari koleksi dengan `friendId` yang baru saja ditangkap.

Line 34: Membuka struktur percabangan evaluasi `if (friend != null) {` untuk memastikan keselamatan sistem dengan memvalidasi keberhasilan proses pencarian data sebelum mencoba menampilkannya.

Line 35 - 37: Mengikat data mutakhir (properti objek `friend`) ke elemen antarmuka grafis yang relevan menggunakan perantara `ViewBinding`:

- Menugaskan nilai `friend.name` ke atribut teks pada komponen `tvDetailName`.
- Menugaskan nilai `friend.date` ke atribut teks pada komponen `tvDetailDate`.
- Menugaskan nilai `friend.feature` ke atribut teks pada komponen `tvDetailFeature`.

Line 39 - 41: Memanfaatkan pustaka `Glide` untuk memproses pemuatan grafis. Rantai instruksi ini meminta konteks dari fragment saat ini (`this`), memuat lokasi sumber daya gambar melalui `friend.photoResId`, dan secara efisien menyuntikkannya ke dalam target komponen gambar `binding.ivDetailPhoto`.

Line 42: Mendeklarasikan blok `} else {` yang berfungsi sebagai penanganan kesalahan visual (error handling) ketika evaluasi data `friend` ternyata mendeteksi nilai kosong (`null`).

Line 43: Menginstruksikan penyematan teks peringatan statis "Data tidak ditemukan" pada komponen judul layar `binding.tvDetailName`.

Line 44 - 45: Rangkaian penutup kurung kurawal `}` yang secara hierarkis menutup logika blok `else` dan menutup penyelesaian pengkondisian percabangan `if`.

Line 47: Melakukan penimpaan terhadap fase destruksi `onDestroyView()`. Siklus ini dipanggil sistem tatkala tampilan visual penunjang `Fragment` dicabut dari memori layar.

Line 48: Memanggil fungsi `super.onDestroyView()` untuk menjamin rutinitas pembersihan sampah memori (garbage collection) kelas induk berjalan sesuai standar ekosistem Android.

Line 49: Secara eksplisit mengubah referensi penyimpanan variabel `_binding` kembali menjadi null. Ini adalah prosedur standar krusial untuk memutus ikatan memori dan mencegah kebocoran (memory leak), mengingat view fisik dari fragment tersebut telah dihancurkan oleh sistem.

Line 50 - 51: Tanda kurung kurawal tutup `}` yang menandakan berakhirnya siklus deklarasi metode `onDestroyView()` dan secara komprehensif mengakhiri ruang lingkup arsitektur kelas `DetailFragment`.

FriendListFragment

line 1: Mendeklarasikan package `com.example.modul3xml.ui` yang menempatkan lokasi file ini secara struktural di dalam direktori khusus antarmuka pengguna proyek Android.

Line 3 - 16: Kumpulan deklarasi import untuk menyertakan antarmuka pemrograman (API) dan pustaka esensial, yang meliputi:

- `android.os.Bundle` dan `android.view.*`: Kelas untuk pertukaran data antar-komponen dan manajemen tata letak hierarki visual.
- `androidx.fragment.app.Fragment` dan `viewModels`: Superkelas pondasi `Fragment` serta utilitas delegasi inisialisasi `ViewModel`.
- `androidx.lifecycle.*`: Kumpulan utilitas reaktif (`Lifecycle`, `lifecycleScope`, `repeatOnLifecycle`) untuk menangani proses asynchronous yang selaras dengan daur hidup aplikasi.
- `androidx.navigation.fragment.findNavController`: Utilitas pengontrol navigasi antar-jendela grafis.
- `androidx.recyclerview.widget.LinearLayoutManager`: Kelas pengatur geometri tata letak linier pada komponen `RecyclerView`.
- `com.example.modul3xml.databinding.FragmentFriendListBinding`: Kelas pengikat tampilan yang menautkan referensi kode ke struktur XML `fragment_friend_list.xml`.

- `kotlinx.coroutines.launch`: Utilitas peluncur utas ringan (coroutine) untuk operasi non-pemblokiran.

Line 18: Mendeklarasikan kelas `FriendListFragment` yang secara absolut mewarisi seluruh kapabilitas siklus hidup grafis dari kelas `Fragment()`.

Line 20: Mendeklarasikan variabel internal `_binding` dengan tipe rujukan `FragmentFriendListBinding?` yang bersifat nullable, dan diinisiasi dengan nilai null untuk merepresentasikan status belum terikat ke memori grafis layar.

Line 21: Mendeklarasikan properti publik binding melalui teknik custom getter (`get() = _binding!!`). Tanda asersi non-null (!!) ini mengonfirmasi kepada kompilator bahwa properti ini hanya akan diakses secara logis pada rentang waktu di mana `_binding` dipastikan sudah memiliki wujud di memori grafis.

Line 24 - 26: Melakukan deklarasi dan instansiasi variabel `viewModel` dengan mendelegasikan proses pembuatannya ke utilitas `by viewModels`. Di dalam blok delegasi ini, sistem diinstruksikan untuk memanggil kelas cetakan kustom `FriendViewModelFactory("Fathi")` agar `ViewModel` dapat terinisialisasi secara tepat dengan memasukkan parameter nama ke dalam konstruktornya.

Line 28 - 31: Melakukan penimpaan (override) metode siklus hidup standar `onCreateView()`. Fase ini adalah tahap krusial di mana kerangka visual awal dari `Fragment` dijahit ke dalam wujud grafis operasional.

Line 32: Mengeksekusi ekspansi (inflation) struktur komponen antarmuka dari bahasa XML menjadi objek manipulatif secara digital (binding) memanfaatkan perantara inflater, kemudian menyimpan hasilnya di variabel internal `_binding`.

Line 33: Mengembalikan nilai akar dari simpul tampilan utama (`binding.root`) kepada sistem agar siap disematkan ke kanvas perangkat keras pengguna.

Line 34: Tanda kurung kurawal tutup `}` untuk menandakan rampungnya eksekusi prosedural di dalam area fungsi `onCreateView()`.

Line 36 - 37: Melakukan penimpaan pada metode `onViewCreated()` serta mendelegasikan rutinitas dasar sistem dengan `super.onViewCreated()`. Fase sekunder ini menjamin

keselamatan operasional saat melakukan intervensi UI, karena hierarki grafis dipastikan telah tercipta dengan sempurna.

Line 40: Menjalankan dan meluncurkan operasi utas ringan yang terisolasi secara asinkron menggunakan `.launch` di dalam lingkup memori khusus milik layar grafis terkait (`viewLifecycleOwner.lifecycleScope`).

Line 41: Memanggil utilitas penahanan eksekusi `repeatOnLifecycle(Lifecycle.State.STARTED)` untuk mengamankan manajemen sumber daya. Fungsi ini memastikan bahwa blok operasi pengamatan data (observasi) hanya diaktifkan jika jendela aplikasi berada di area pandang pengguna (minimal berstatus `STARTED`).

Line 43: Mengeksekusi penyerapan aliran data (`.collect`) dari sumber reaktif `viewModel.friendsState`. Operasi ini menempatkan keluaran aliran data mutakhir ke dalam referensi variabel kolektif berwujud `friendsList`.

Line 46: Mengonfigurasi mekanisme tata letak parameter `binding.rvFeaturedFriends.layoutManager` menggunakan `LinearLayoutManager` dan memodifikasinya secara eksplisit menjadi orientasi `HORIZONTAL` agar barisan daftar dapat digulir ke area kanan dan kiri layar.

Line 47: Melakukan instansiasi pemetaan adapter terhadap daftar `binding.rvFeaturedFriends.adapter`. Sistem memanggil kelas `FriendAdapter` sembari memotong jumlah entitas yang diterima dengan ekspresi rantai `friendsList.take(5)`, sehingga menjamin hanya lima objek indeks pertama yang diserahkan. Di ujung fungsi, lambda expression digunakan untuk mendefinisikan tanggapan apabila interaksi klik terjadi atas `selectedFriend`.

Line 49: Memicu pemanggilan instrumen rekam diagnostik melalui metode `viewModel.onFriendClicked(selectedFriend)`, yang bertugas melakukan penetrasi jejak aktivitas operasional navigasi di konsol utama log.

Line 52: Membuat bundel paket data (`Bundle().apply`) yang dikonfigurasi khusus untuk menjejalkan pengenalan data bilangan bulat (`putInt`) dengan label kunci `"friendId"`, berisi nilai absolut ID identitas yang diklik (`selectedFriend.id`).

Line 53: Mengeksekusi transmisi perpindahan grafis (navigasi) dengan melibatkan instruktur sistem navigasi (`findNavController()`) menuju koordinat destinasi berlabel `action_friendListFragment_to_detailFragment`, sekaligus mendiktekan pembawaan bungkusan `Bundle` tadi.

Line 54: Tanda kurung kurawal tutup `}` untuk menandai berakhirnya algoritma logika operasional pada tindakan klik untuk kelompok adapter susunan lintasan horizontal.

Line 57: Mengonfigurasi mekanisme tata letak parameter sekunder pada `binding.rvAllFriends.layoutManager` menggunakan instansiasi ulang kelas `LinearLayoutManager` pada konfigurasi poros bawaan sistem (`VERTICAL`).

Line 58: Melakukan pembentukan ulang instansiasi `FriendAdapter` sebagai subjek delegasi perenderan pada kelompok grafis daftar utama. Fungsi ini memasukkan seluruh volume inventori data `friendsList` (tanpa pembatasan) beserta fungsionalitas deklarasi aksi klik.

Line 60: Merupakan pemanggilan identik logik ke fasilitas diagnostik `log viewModel.onFriendClicked(selectedFriend)` bagi setiap komponen tunggal yang mengalami aktivasi pada layar daftar linier vertikal.

Line 63: Menciptakan konfigurasi bundel penyimpanan rute navigasi `friendId` bagi penanganan layar rincian detail secara dinamis.

Line 64: Memerintahkan kontroler navigasi internal untuk mendelegasikan peralihan transisi jendela visual antarmuka sistem layar Detail yang setara dengan penanganan pada orientasi layar sejajar.

Line 65 - 69: Rangkaian rentetan kurung kurawal penutup `}` yang menyegel batas zona hierarkis secara bertingkat: blok tanggapan antarmuka klik, ruang pengamatan data sinkron (kolektor aliran reaktif), blok siklus aman status pemantauan grafis, hingga pilar pengakhiran area kerja peluncur utas kustom `coroutine`.

Line 71 - 72: Melakukan pendefinisian blok rutinitas standar metode `onDestroyView()` untuk memitigasi anomali eksekusi pasca-kehancuran tampilan layar, sembari merujuk penyelesaian ke tumpukan metode kelas utama (`super.onDestroyView()`).

Line 73: Memberikan penugasan operasional vital terhadap alokator alamat `_binding` dengan memberikan besaran mutlak bernilai null. Implementasi pencegahan destruktif ini berperan mutlak agar Garbage Collector dapat menghancurkan file grafis tanpa dihalangi referensi memori yang nyangkut (memory-leak prevention).

Line 74 - 75: Merupakan sepasang simpul akhir elemen kurung kurawal `}` yang mendefinisikan pembatasan penutupan siklus rutinitas `onDestroyView` sekaligus mendemonstrasikan blok pengunci akhir pendefinisian pada arsitektur wujud deklaratif dari entitas kelas dasar `FriendListFragment`.

FriendViewModel.kt

Line 1: Deklarasi package `com.example.modul3xml.ui` yang menentukan ruang lingkup kelas ini di dalam hierarki direktori antarmuka proyek berbasis XML.

Line 3 - 10: Kumpulan instruksi import untuk menyertakan pustaka yang relevan:

- `androidx.lifecycle.ViewModel` dan `ViewModelProvider`: Kelas pondasi untuk membangun arsitektur pelestarian data (`ViewModel`) dan pabrik pembuatnya (`factory`).
- `com.example.modul3xml.data.FriendData` dan `model.Friend`: Mengimpor pangkalan data statis serta cetak biru model struktur data kawan.
- `kotlinx.coroutines.flow.*`: Mengimpor instrumen pemrograman reaktif (`MutableStateFlow`, `StateFlow`, `asStateFlow`) untuk pengelolaan status aliran data asinkron (state management).
- `timber.log.Timber`: Pustaka pihak ketiga untuk pencatatan riwayat sistem (logging).

Line 12: Mendeklarasikan kelas `FriendViewModel` yang mewarisi sifat operasi dari kelas dasar `ViewModel()`. Kelas ini memiliki konstruktor utama yang mengharuskan penyisipan parameter `userName` bertipe data `String`.

Line 15: Mendeklarasikan variabel privat `_friendsState` yang diinisialisasi sebagai `MutableStateFlow` dengan nilai bawaan berupa senarai kosong (`emptyList()`). Status ini bersifat mutable (dapat dimodifikasi datanya) secara eksklusif hanya dari dalam kelas ini.

Line 16: Mendeklarasikan properti publik `friendsState` bertipe `StateFlow<List<Friend>>` yang nilainya direferensikan pada `_friendsState.asStateFlow()`. Pendekatan ini menerapkan prinsip enkapsulasi di mana pihak luar (seperti `Fragment` atau `Activity`) hanya memiliki kewenangan akses baca (read-only) terhadap aliran data.

Line 18: Membuka blok `init {`, yang mana sistem akan mengeksekusi kumpulan instruksi di dalamnya secara mutlak sesaat setelah instansiasi objek `ViewModel` berhasil dibentuk di memori.

Line 19: Memanggil fungsi metode internal `loadFriends()` untuk mengeksekusi proses pemuatan data awal ke dalam aliran reaktif.

Line 20: Memanfaatkan utilitas `Timber.d()` untuk mencetak pesan log ke konsol debug yang memuat informasi parameter nama pengguna (`$userName`) selaku pihak yang menginisialisasi `ViewModel` tersebut.

Line 21: Tanda kurung kurawal tutup `}` yang menandakan rampungnya eksekusi prosedur pada blok inisialisasi `init`.

Line 23: Mendeklarasikan fungsi `loadFriends()` dengan tingkat penentu akses privat (`private`), sehingga fungsi pemuatan ini terlindungi dan tidak dapat dipicu secara sembarangan oleh kelas lain di luar lingkup `ViewModel` ini.

Line 24: Menciptakan variabel operasional lokal `data` untuk menyalin dan menampung seluruh koleksi entitas kawan yang bersumber dari basis data `FriendData.friends`.

Line 25: Mendistribusikan muatan variabel data ke dalam properti penampung `.value` milik aliran reaktif `_friendsState`. Aksi injeksi ini secara instan akan memicu pemberitahuan pembaruan pada pengamat (`observer`) yang sedang memantau data tersebut di lapisan UI.

Line 26: Mencetak pesan diagnostik `Timber.d()` yang mengonfirmasi keberhasilan penambahan entitas ke dalam koleksi, lengkap dengan melampirkan perhitungan kuantitas besaran elemen melalui fungsi `data.size`.

Line 27: Tanda kurung kurawal tutup `}` untuk membatasi akhir dari logika prosedural `loadFriends()`.

Line 30: Mendeklarasikan fungsi perilaku eksternal `onFriendClicked` yang menerima argumen berwujud objek utuh `Friend`. Fungsi publik ini beroperasi sebagai penangkap delegasi logis ketika pengguna berinteraksi (menekan) elemen antarmuka item kawan.

Line 31: Memicu cetakan log diagnostik tingkat pertama yang merekam aktivasi penekanan tombol rincian berserta ekstraksi atribut nama (`friend.name`) dari objek yang sedang berinteraksi.

Line 32: Memicu cetakan log diagnostik tingkat sekunder yang bertugas merekam rekam jejak peralihan halaman, sekaligus memetakan konversi seluruh struktur nilai atribut dari objek `friend` ke dalam bentuk teks.

Line 33: Tanda kurung kurawal tutup `}` untuk mengakhiri pendefinisian instruksi pada ruang lingkup fungsi `onFriendClicked`.

Line 34: Tanda kurung kurawal tutup `}` yang mendefinisikan pembatasan akhir dari ruang lingkup arsitektur secara keseluruhan pada kelas `FriendViewModel`.

Line 36: Mendeklarasikan kelas pendukung bernama `FriendViewModelFactory` yang mewarisi fungsi cetakan antarmuka `ViewModelProvider.Factory`. Kelas ini dirancang secara khusus untuk mendelegasikan proses inisialisasi `ViewModel` yang membutuhkan spesifikasi parameter khusus pada konstruktornya.

Line 37: Mengesampingkan (override) fungsionalitas turunan generik `create()`. Ini adalah metode rujukan standar milik sistem penyedia `ViewModel` untuk membangun serta mengembalikan entitas dari spesifikasi kelas yang disasar.

Line 38: Membangun blok percabangan pengkondisian `if` untuk mengevaluasi parameter kesesuaian turunan (`isAssignableFrom`) mengenai apakah kelas yang direpresentasikan oleh `modelClass` ekuivalen dan sejalan dengan cetakan `FriendViewModel::class.java`.

Line 39: Menambahkan anotasi kompilasi penekan peringatan `@SuppressWarnings("UNCHECKED_CAST")`. Instruksi ini menginformasikan kepada penganalisis kode (linter) untuk mengabaikan peringatan keamanan mengenai pemaksaan transposisi tipe data (type casting), berhubung validasi asuransi rujukan telah dituntaskan secara logis pada percabangan sebelumnya.

Line 40: Menjalankan eksekusi konstruksi terhadap pembentukan objek kelas FriendViewModel secara eksplisit dengan menyisipkan argumen string name, yang kemudian hasil instansiasinya dikembalikan seraya dipaksa bertransformasi sesuai luaran tipe parameter generik as T.

Line 41: Tanda kurung kurawal tutup } yang menandakan berakhirnya rentang blok evaluasi logika kontrol if.

Line 42: Memberlakukan sistem penghentian operasi dengan menjatuhkan lontaran eksepsi IllegalArgumentException("Unknown ViewModel class"). Pernyataan ini diprogram sebagai langkah mitigasi keamanan preventif yang akan menggagalkan kompilasi sistem saat factory diminta untuk membentuk entitas referensi kelas yang belum didefinisikan.

Line 43: Tanda kurung kurawal tutup } untuk merampungkan batas operasional pada blok metode konstruksi create().

Line 44: Tanda kurung kurawal tutup } final yang berfungsi untuk menyegel keutuhan struktur definisi pada arsitektur kelas penyedia FriendViewModelFactory.

MainActivity.kt

Line 1: Deklarasi package com.example.modul3xml yang berfungsi sebagai identitas unik aplikasi dan menentukan lokasi struktural file ini di dalam ekosistem direktori proyek Android.

Line 3 - 4: Kumpulan instruksi import untuk menyertakan pustaka dasar bawaan sistem Android:

- android.os.Bundle: Kelas pengemas yang digunakan untuk menyimpan dan memulihkan status data aplikasi ketika terjadi perubahan konfigurasi sistem (seperti transisi rotasi layar).
- androidx.appcompat.app.AppCompatActivity: Kelas dasar (superclass) standar untuk komponen aktivitas di Android modern. Kelas ini memberikan dukungan kompatibilitas mundur (backward compatibility) agar fitur-fitur antarmuka material terbaru dapat berjalan stabil pada versi Android yang lebih usang.

Line 6: Mendeklarasikan kelas MainActivity yang secara absolut mewarisi seluruh sifat dan fungsi dari superkelas AppCompatActivity(). Kelas ini dikonfigurasi sebagai titik masuk utama (entry point) atau jendela pertama yang akan diluncurkan oleh sistem saat pengguna mengetuk ikon aplikasi.

Line 7: Melakukan penimpaan (override) pada metode siklus hidup fundamental onCreate(). Fungsi ini adalah tahapan inisialisasi paling awal yang dipicu secara otomatis oleh sistem operasi Android saat instansiasi aktivitas mulai diciptakan di memori. Parameter savedInstanceState bertipe Bundle? (nullable) bertugas untuk menangkap data rekam jejak layar sebelumnya apabila aplikasi pernah dihentikan oleh sistem untuk membebaskan memori.

Line 8: Mengeksekusi pemanggilan metode super.onCreate(savedInstanceState) untuk mendelegasikan instruksi kepada hierarki kelas induk. Langkah ini diwajibkan agar kerangka kerja sistem Android dapat menuntaskan proses persiapan lingkungan dasar (environment setup) sebelum mengeksekusi modifikasi tambahan.

Line 9: Mengeksekusi fungsi setContentView(R.layout.activity_main). Perintah vital ini bertugas menjahit kelas logika Kotlin dengan berkas rancangan antarmuka pengguna statis berbasis XML (activity_main.xml). Sistem akan memproses (inflate) rancangan XML tersebut dan memproyeksikannya menjadi elemen visual antarmuka grafis yang nyata di layar perangkat keras.

Line 10: Tanda kurung kurawal tutup } yang menandakan rampungnya eksekusi prosedural di dalam area blok metode onCreate().

Line 11: Tanda kurung kurawal tutup } final yang menyegel batas ruang lingkup operasional dan mengakhiri pendefinisian arsitektur struktural pada kelas MainActivity secara menyeluruh.

MainApplication.kt

Line 1: Mendeklarasikan package com.example.modul3xml yang berfungsi menetapkan ruang lingkup dan identitas file ini di dalam struktur direktori proyek aplikasi Android.

Line 3 - 4: Mengimpor pustaka dasar tingkat sistem yang diwajibkan oleh file ini, meliputi:

- `android.app.Application`: Superkelas bawaan ekosistem Android yang merepresentasikan dan mengelola status global dari seluruh aplikasi.
- `timber.log.Timber`: Modul pustaka pihak ketiga yang digunakan sebagai utilitas pencatatan log (perekaman riwayat diagnostik sistem) terotomatisasi.

Line 6: Mendeklarasikan kelas `MainApplication` yang secara eksplisit mewarisi kapabilitas dari superkelas `Application()`. Kelas ini dikonfigurasi untuk menjadi entitas pertama yang diinisialisasi oleh sistem operasi Android, bahkan sebelum komponen antarmuka atau aktivitas (`Activity`) apa pun diluncurkan ke layar.

Line 7: Melakukan penimpaan (`override`) pada metode siklus hidup `onCreate()`. Fase fundamental ini dipanggil oleh sistem secara otomatis tepat satu kali saat kerangka utama aplikasi berhasil dialokasikan ke dalam memori perangkat.

Line 8: Mengeksekusi metode `super.onCreate()` guna mendelegasikan instruksi kepada kelas induk. Langkah ini memastikan bahwa sistem Android menyelesaikan seluruh inisialisasi lingkungan dasar aplikasi dengan benar sebelum mengeksekusi instruksi konfigurasi kustom.

Line 11: Mendeklarasikan variabel konstan `isDebuggable` untuk mengevaluasi status kompilasi aplikasi. Sistem melakukan operasi logika bitwise `and` antara atribut bendera status aplikasi saat ini (`applicationInfo.flags`) dengan bendera absolut referensi sistem Android (`ApplicationInfo.FLAG_DEBUGGABLE`) guna memvalidasi secara logis apakah kerangka kerja sedang beroperasi dalam mode pengembangan (`debug mode`) atau telah didistribusikan secara komersial (`production mode`).

Line 12: Membuka blok percabangan pengkondisian `if (isDebuggable)` {. Blok ini bertindak sebagai mekanisme keamanan (`safeguard`) logis untuk memastikan bahwa instruksi pelacakan di dalamnya hanya akan tereksekusi apabila aplikasi sedang berada dalam tahap `debugging`.

Line 13: Mengeksekusi penanaman konfigurasi (`planting`) instrumen pencatat log dengan memanggil `Timber.plant(Timber.DebugTree())`. Konfigurasi ini menghidupkan mesin logger

yang akan secara otomatis menangkap, memformat, dan menyiarkan seluruh jejak variabel serta aktivitas sistem langsung ke konsol Logcat selama proses pengembangan.

Line 14: Tanda kurung kurawal tutup } yang mendefinisikan batas akhir dari ruang lingkup eksekusi pada kondisi percabangan pengecekan status debug.

Line 15: Tanda kurung kurawal tutup } untuk merampungkan rentetan instruksi operasional pada metode inisialisasi onCreate().

Line 16: Tanda kurung kurawal tutup } final yang menyegel batas ruang lingkup operasional dan mengakhiri pendefinisian arsitektur struktural pada kelas MainApplication secara menyeluruh.

Fragment_detail.xml

Line 1: <?xml version="1.0" encoding="utf-8"?>. Baris standar yang memberi tahu sistem bahwa ini adalah file berekstensi XML dengan format teks utf-8.

Line 2 - 4: Membuka komponen ScrollView. Ini adalah wadah wajib agar halaman detail ini bisa digeser (di-scroll) ke bawah seandainya cerita temanmu sangat panjang. Tiga baris xmlns (XML namespace) mendaftarkan aturan penulisan android, app, dan tools.

Line 5 - 6: Mengatur ukuran ScrollView agar lebar (width) dan tingginya (height) melar memenuhi layar (match_parent).

Line 7: tools:context=".ui.DetailFragment">. Menandakan bahwa desain ini adalah pasangan dari file Kotlin DetailFragment.kt. Tanda > menutup pembukaan ScrollView.

Line 9: Membuka komponen LinearLayout. Karena ScrollView hanya boleh berisi satu anak saja, kita masukkan LinearLayout ini sebagai "keranjang" untuk menaruh gambar dan teks-teks di bawahnya.

Line 10 - 11: Lebar keranjang dibuat selebar layar (match_parent), tapi tingginya dibuat mengikuti isi di dalamnya saja (wrap_content).

Line 12: android:orientation="vertical". Ini aturan paling penting. Ia memaksa semua barang di dalam keranjang ini (foto, nama, tanggal, dll) disusun berbaris dari atas ke bawah.

Line 13: `android:padding="16dp">`. Memberi bantalan/jarak aman di sekeliling keranjang agar isinya tidak menempel ke tepi pinggir layar HP. Tanda `>` menutup pembukaan `LinearLayout`.

Line 15 - 20: Komponen `ShapeableImageView` untuk menampilkan foto profil besar.

- Line 16: Memberi ID `ivDetailPhoto`.
- Line 17 - 18: Lebarnya penuh (`match_parent`), tingginya dikunci di 350dp.
- Line 19: `scaleType="centerCrop"` memotong pinggiran gambar agar pas di kotak tanpa gepeng.

Line 20: `app:shapeAppearanceOverlay="@style/RoundedImage" />` menerapkan gaya rahasia yang melengkungkan sudut gambar. Tanda `/>` menutup komponen gambar.

Line 22 - 29: Komponen `TextView` untuk nama. Diberi ID `tvDetailName`, jarak ke atas 16dp, ukuran teks besar 28sp, dan dicetak tebal (bold). Perintah `tools:text="Nama Teman"` hanya memunculkan teks palsu saat kamu melihat preview desain di Android Studio.

Line 31 - 39: Komponen `TextView` untuk tanggal. Diberi ID `tvDetailDate`, ukuran 16sp, dan diwarnai abu-abu gelap menggunakan kode hex `#666666`. Diberi margin atas dan bawah agar posisinya pas.

Line 41 - 46: Komponen `TextView` untuk label statis. Karena tidak perlu diubah dari Kotlin, komponen ini tidak perlu ID. Isinya langsung diisi `android:text="Feature:"` dengan ukuran 18sp dan cetak tebal.

Line 48 - 55: Komponen `TextView` untuk cerita lengkap. Diberi ID `tvDetailFeature`. Lebarnya dibuat mentok kiri-kanan (`match_parent`), ukuran teks 16sp, dan diberi `lineSpacingExtra="8sp"` agar jarak antar barisnya lebih renggang dan nyaman dibaca.

Line 57: `</LinearLayout>`. Tag penutup untuk mengakhiri keranjang `LinearLayout`.

Line 58: `</ScrollView>`. Tag penutup terakhir untuk mengakhiri wadah layar `ScrollView`.

Fragment_friend_list.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Baris standar yang selalu ada di awal untuk menandakan ini adalah dokumen XML.

Line 2 - 7: Membuka wadah paling luar yaitu `NestedScrollView`. Kenapa tidak pakai `LinearLayout` biasa? Karena daftar temanmu mungkin sangat panjang, wadah ini memastikan seluruh area halaman bisa digeser (di-scroll) ke bawah. Baris `tools:context` di sini menyambungkan desain ini dengan file `Kotlin FriendListFragment`.

Line 9 - 13: Membuka komponen `LinearLayout`. Aturan di Android, wadah scroll hanya boleh memiliki satu anak langsung. Jadi kita butuh `LinearLayout` ini sebagai "keranjang" utamanya. Kita atur `orientation="vertical"` agar isi keranjang ini berbaris lurus dari atas ke bawah, dan `paddingBottom="16dp"` agar bagian paling bawah tidak terlalu menempel dengan dasar HP.

Line 15 - 21: Komponen `TextView` untuk judul rak pertama. Menampilkan teks "Featured Friends" dengan ukuran besar (20sp), dicetak tebal (bold), dan diberi jarak margin 16dp di sekelilingnya agar posisinya pas.

Line 23 - 30: Komponen `RecyclerView` untuk rak atas (geser menyamping).

Line 24: Diberi ID `rvFeaturedFriends` agar mudah dipanggil dari Kotlin.

Line 27: `clipToPadding="false"`. Ini trik rahasia desainer UI/UX! Ini membuat efek visual yang cantik saat kartu digeser, karena kartu tidak akan terpotong secara kaku oleh garis batas layar.

Line 28: `orientation="horizontal"`. Baris inilah yang mengubah sifat bawaan daftar dari yang tadinya ke bawah, menjadi berjejer menyamping.

Line 30: `app:layoutManager="..."` memberi tahu Android untuk menyusun isinya secara berurutan (linear).

Line 32 - 38: Komponen `TextView` untuk judul rak kedua. Sama seperti di atas, ini menampilkan tulisan "All Friends" sebagai pembatas sebelum masuk ke daftar yang utuh.

Line 40 - 45: Komponen `RecyclerView` untuk rak bawah (geser ke bawah).

Line 41: Diberi ID `rvAllFriends`.

Line 44: `android:nestedScrollingEnabled="false"`. halaman ini dibungkus oleh `NestedScrollView` (yang fungsinya buat scroll), kalau kita tidak mematikan fungsi scroll bawaan dari `RecyclerView` ini, akan nge-bug atau macet karena ada dua perintah scroll yang saling bentrok.

Line 47: `</LinearLayout>`. Tag penutup untuk mengakhiri keranjang penyusun vertikal.

Line 48: `</androidx.core.widget.NestedScrollView>`. Tag penutup paling akhir untuk mengakhiri wadah layar yang bisa digeser tersebut.

Item_friend.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Baris pembuka standar yang menandakan dokumen ini menggunakan bahasa XML.

Line 2 - 10: Membuka komponen `MaterialCardView`. Ini adalah wadah paling luar yang berfungsi sebagai "bingkai kartu" untuk profil teman. Kita menggunakan komponen ini agar kartunya bisa diberi lengkungan di sudut (`cardCornerRadius="16dp"`) dan diberi efek bayangan tipis (`cardElevation="2dp"`) agar terlihat seperti 3D.

Line 12 - 17: Membuka `LinearLayout` pertama (Utama). Diatur dengan `orientation="horizontal"`. Ini adalah keranjang utama di dalam kartu yang membagi isi menjadi kiri dan kanan (foto di kiri, teks di kanan). Diberi jarak ke dalam (`padding="16dp"`) agar isinya tidak menempel ke garis tepi kartu.

Line 19 - 24: Komponen `ShapeableImageView`. Ini berfungsi untuk menampilkan foto profil teman. Ukurannya dikunci menjadi kotak pas 100dp x 100dp. Aturan `centerCrop` memastikan foto tidak gepeng, dan `shapeAppearanceOverlay` memanggil gaya agar sudut foto ikut melengkung serasi dengan kartunya.

Line 26 - 31: Membuka `LinearLayout` kedua. Diatur dengan `orientation="vertical"`. Ini adalah keranjang khusus untuk menumpuk teks dan tombol dari atas ke bawah. Penggunaan `layout_weight="1"` di sini sangat penting; fungsinya menyuruh keranjang teks ini melar mengisi semua sisa ruang kosong di sebelah kanan foto profil.

Line 33 - 38: Membuka LinearLayout ketiga (Horizontal). Ini adalah keranjang kecil yang khusus dibuat hanya untuk menjejerkan baris Nama dan Tanggal secara bersebelahan.

Line 40 - 48: Komponen TextView untuk Nama. Diberi `layout_weight="1"` agar nama ini mengambil jatah ruang lebih banyak dan mendorong posisi teks tanggal mentok ke ujung kanan. Diberi batas maksimal 2 baris, dan jika namanya sangat panjang, akan dipotong halus dengan titik-titik di akhirnya (`ellipsize="end"`). (Catatan: Di kodemu, tag penutup teks nama dan pembuka teks tanggal menyambung di baris yang sama).

Line 48 - 54: Komponen TextView untuk Tanggal. Ukurannya lebih kecil (10sp) dan diberi jarak margin di sisi kiri (`marginStart`) agar tidak menempel dengan teks nama.

Line 55: `</LinearLayout>`. Tag penutup untuk mengakhiri keranjang kecil pembungkus baris nama dan tanggal.

Line 57 - 65: Komponen TextView untuk Deskripsi (`tvFeature`). Sama seperti nama, deskripsi ini dibatasi maksimal 2 baris agar kartu tidak menjadi terlalu panjang ke bawah, dan sisa teksnya disembunyikan menggunakan efek titik-titik (`ellipsize="end"`).

Line 67 - 72: Membuka LinearLayout keempat (Horizontal). Keranjang terakhir ini bertugas khusus untuk menjejerkan tombol-tombol di bagian paling bawah.

Line 74 - 80: Komponen Button untuk Instagram. Diberi ukuran lebar 0dp dan `layout_weight="1"`.

Line 82 - 89: Komponen Button untuk Detail. Ini juga diberi lebar 0dp dan `layout_weight="1"`. Karena kedua tombol ini menggunakan `weight` (bobot) yang sama yaitu 1, maka sistem akan secara otomatis membagi ruang keduanya persis rata 50:50.

Line 90 - 94: Kumpulan tag penutup. Baris-baris ini bertugas menutup semua keranjang LinearLayout yang bersarang tadi, dan diakhiri dengan penutup bingkai utama `MaterialCardView`.

Nav_graph.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Ini adalah baris wajib pembuka yang memberi tahu komputer bahwa dokumen ini ditulis dalam bahasa XML.

Line 2 - 4: Membuka komponen `<navigation>`. Ini adalah lembaran peta utamanya. Tiga baris `xmlns` di bawahnya bertugas mendaftarkan kamus perintah agar sistem Android paham instruksi yang akan kita tulis.

Line 5: `android:id="@+id/nav_graph"`. Memberikan nama ID untuk peta jalan ini agar mudah dikenali oleh sistem.

Line 6: `app:startDestination="@id/friendListFragment">`. Baris ini sangat penting! Ini berfungsi sebagai "titik start". Kita menyuruh aplikasi untuk selalu membuka halaman `friendListFragment` paling pertama saat aplikasi baru dijalankan.

Line 8 - 12: Mendaftarkan titik lokasi (layar) pertama, yaitu `<fragment>`.

Line 9: Diberi ID `friendListFragment`.

Line 10: Disambungkan dengan file Kotlin `FriendListFragment` yang menjadi otak dari layar ini.

Line 11: Diberi label atau judul layar "Friend List".

Line 12: Disambungkan dengan file desain `fragment_friend_list.xml` agar kita bisa melihat preview visualnya di peta.

Line 14 - 16: Membangun jalan penghubung (`<action>`). Ini adalah rute perpindahan. Kita memberi ID untuk rute ini (`action_friendListFragment_to_detailFragment`), lalu mengarahkan tujuan akhir rute tersebut (destination) ke halaman detail. Rute inilah yang nanti akan kita panggil dari file Kotlin saat tombol ditekan.

Line 17: `</fragment>`. Tag penutup untuk mengakhiri pendaftaran layar pertama.

Line 19 - 23: Mendaftarkan titik lokasi (layar) kedua. Caranya sama persis dengan yang di atas. Diberi ID `detailFragment`, disambungkan ke Kotlin `DetailFragment`, diberi judul "Detail Friend", dan disambungkan ke file desain `fragment_detail.xml`.

Line 25 - 28: Membuka loket penerimaan paket (`<argument>`). Karena halaman detail ini butuh data dari halaman sebelumnya, kita harus mendaftarkan paket apa yang akan masuk.

Line 26: Paketnya diberi nama friendId.

Line 27: Tipe isi paketnya adalah integer (angka bilangan bulat).

Line 28: Jika ternyata ada error dan paketnya kosong, nilai cadangannya disetel menjadi 0 (defaultValue="0").

Line 29: </fragment>. Tag penutup untuk mengakhiri pendaftaran layar kedua.

Line 31: </navigation>. Tag penutup paling akhir untuk melipat dan menutup keseluruhan dokumen peta navigasi ini.

Pembahasan Compose

MainActivity.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu com.example.modul4compose. Ini berfungsi sebagai identitas unik aplikasimu dan menentukan di folder mana file ini tersimpan di dalam sistem.

Line 3 - 12: Bagian ini bertugas memanggil berbagai alat (library) dari Android dan Jetpack Compose yang dibutuhkan oleh file ini, seperti alat untuk membuat aktivitas, alat pendesain layar, hingga memanggil tema dan rute navigasi yang sudah kamu buat.

Line 14: Mendeklarasikan class MainActivity yang mewarisi sifat dari ComponentActivity(). Class ini adalah jendela utama aplikasi Android yang dirancang untuk komponen antarmuka modern (Jetpack Compose).

Line 15: Mendefinisikan ulang (override) fungsi onCreate(). Fungsi ini adalah hal pertama yang otomatis dibangun dan dijalankan oleh sistem operasi Android ketika pengguna mengetuk ikon aplikasi di layar HP.

Line 16: Memanggil super.onCreate(savedInstanceState) untuk memastikan sistem dasar Android mempersiapkan memori, status, dan jendela aplikasi dengan benar sebelum kita memasukkan desain kita sendiri.

Line 17: Memanggil fungsi enableEdgeToEdge(). Perintah ini membuat tampilan aplikasi melebar penuh secara imersif, menembus area baris baterai/sinyal (status bar) di atas dan area tombol navigasi di bawah.

Line 18: Memanggil fungsi `setContent { ... }`. Fungsi ini bertugas sebagai jembatan yang mengubah peran dari pengaturan sistem Android menjadi "papan gambar". Semua kode desain tampilan aplikasi akan digambar di dalam blok ini.

Line 19: Memanggil `Modul3ComposeTheme { ... }` untuk membungkus seluruh desain UI. Dengan cara ini, semua komponen di dalam aplikasi akan secara otomatis mengikuti gaya desain, warna, dan jenis huruf yang seragam sesuai tema.

Line 20: Memanggil `Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->.` `Scaffold` adalah kerangka dasar layar yang dimaksimalkan ukurannya hingga memenuhi layar. Kerangka ini menghitung batas area aman layar dan menghasilkan nilai `innerPadding` agar konten tidak tertutup oleh bagian atas layar atau tombol navigasi HP.

Line 22: Memanggil komponen `AppNavigation` (yang merupakan fungsi buatanmu di file lain. Fungsi ini memegang kendali penuh atas peta jalan aplikasi (berisi daftar teman dan halaman detail).

Line 23: Meneruskan nilai `padding (modifier = Modifier.padding(innerPadding))` dari `Scaffold` ke dalam `AppNavigation`. Tujuannya sangat penting, yaitu agar seluruh layar di dalam navigasi tersebut tahu batas area aman dan kontennya tidak terpotong sistem HP.

Line 24: Tanda kurung tutup `)` untuk mengakhiri pengiriman pengaturan (parameter) pada komponen `AppNavigation`.

Line 25 - 29: Barisan kurung kurawal tutup `}` ini secara berurutan berfungsi untuk menutup blok `Scaffold` (Line 25), menutup blok Tema (Line 26), menutup blok `setContent` (Line 27), menutup fungsi `onCreate` (Line 28), dan terakhir menutup kelas `MainActivity` (Line 29).

FriendData

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul3compose.data`. Package ini menandakan bahwa file ini berada di dalam folder data, yang umumnya digunakan untuk memisahkan logika penyediaan data dari tampilan antarmuka (UI).

Line 2: Baris kosong sebagai pemisah antara deklarasi package dan blok import.

Line 3 - 4: Mengimpor file dan class yang dibutuhkan:

- `com.example.modul3compose.R`: digunakan untuk mengakses *resource* bawaan Android, seperti gambar yang tersimpan di dalam folder `drawable`.
- `com.example.modul3compose.model.Friend`: mengimpor *data class* `Friend` yang berfungsi sebagai cetakan atau model struktur data untuk setiap objek teman.

Line 5: Baris kosong sebagai pemisah.

Line 6: Mendeklarasikan object `FriendData`. Di dalam Kotlin, penggunaan kata kunci object menandakan pembuatan sebuah *Singleton*, artinya class ini hanya memiliki satu *instance* di seluruh siklus hidup aplikasi dan isinya bisa langsung diakses tanpa perlu membuat objek baru.

Line 7: Mendeklarasikan variabel `friends` yang bernilai `listOf(...)`. Ini berarti variabel `friends` menyimpan koleksi data berupa *List* yang bersifat *read-only* (tidak bisa ditambah/dikurangi setelah dibuat), di mana setiap elemen di dalamnya adalah instansiasi dari class `Friend`.

Line 8 - 15: Merupakan instansiasi objek `Friend` yang pertama di dalam *list*. Objek ini diisi dengan data *dummy* menggunakan parameter yang didefinisikan di data class `Friend`:

- `id = 1`: identitas unik dari data ini.
- `name`: menyimpan nama teman.
- `date`: menyimpan data tanggal atau keterangan waktu.
- `feature`: deskripsi panjang atau cerita lucu terkait teman tersebut.
- `photoResId = R.drawable.lv`: memanggil *resource* gambar bernama `lv` yang ada di dalam folder `res/drawable`.
- `instagramUrl`: tautan URL menuju profil Instagram.
- Line 15 ditutup dengan `)`, yang menandakan akhir dari objek pertama dan bersiap menerima objek kedua.

Line 16 - 23: Merupakan instansiasi objek `Friend` yang kedua, dengan `id = 2` atas nama "Fatih Bari Muar". Struktur pengisian propertinya (*name*, *date*, *feature*, gambar `R.drawable.fatih`, dan URL) sama persis dengan pola pada objek pertama.

Line 24 - 31: Merupakan instansiasi objek Friend yang ketiga, dengan id = 3 atas nama "Meluyyyyyyy". Menggunakan gambar R.drawable.melon.

Line 32 - 39: Merupakan instansiasi objek Friend yang keempat, dengan id = 4 atas nama "neRvers.". Menggunakan gambar R.drawable.nafis.

Line 40 - 47: Merupakan instansiasi objek Friend yang kelima sekaligus yang terakhir, dengan id = 5 atas nama "Rakha x Police". Menggunakan gambar R.drawable.rakha. Pada baris ke-47, ditutup dengan) tanpa tanda koma di belakangnya karena ini adalah elemen terakhir di dalam listOf.

Line 48: Kurung tutup) untuk mengakhiri fungsi listOf pada variabel friends.

Line 49: Kurung kurawal tutup } untuk mengakhiri blok deklarasi object FriendData.

Friend

Line 1: Merupakan deklarasi package file Kotlin yaitu com.example.modul3compose.model. Penamaan model di ujung package menandakan bahwa file ini diletakkan khusus di dalam folder yang tugasnya hanya untuk mendefinisikan struktur data.

Line 2: Baris kosong sebagai pemisah.

Line 3: Mengimpor anotasi androidx.annotation.DrawableRes. Anotasi ini nantinya digunakan untuk memberi tahu sistem Android bahwa sebuah variabel angka (Integer) adalah sebuah ID gambar dari folder drawable.

Line 4: Baris kosong sebagai pemisah antara blok import dan deklarasi class.

Line 5: Mendeklarasikan data class Friend(. Di dalam Kotlin, data class adalah tipe class yang secara khusus dirancang hanya untuk menyimpan data. Dengan memakai data class, Kotlin otomatis membuatkan fungsi-fungsi berguna di belakang layar (seperti membandingkan atau menyalin data) tanpa perlu kita coding manual. Tanda kurung buka (menandakan awal dari daftar data apa saja yang dimiliki oleh class ini.

Line 6: Mendeklarasikan properti id dengan tipe data Int (angka bulat). Ini digunakan sebagai identitas unik atau nomor urut untuk setiap teman.

Line 7: Mendeklarasikan properti name dengan tipe data String (teks) untuk menyimpan nama teman.

Line 8: Mendeklarasikan properti date dengan tipe data String untuk menyimpan tanggal atau keterangan waktu terkait teman tersebut.

Line 9: Mendeklarasikan properti feature dengan tipe data String untuk menyimpan deskripsi, cerita, atau keterangan panjang tentang teman tersebut.

Line 10: Mendeklarasikan properti photoResId dengan tipe data Int. Terdapat anotasi `@DrawableRes` di depannya sebagai penanda khusus agar Android Studio bisa mengecek dan mencegah error jika kita salah memasukkan angka biasa yang bukan merujuk ke ID gambar.

Line 11: Mendeklarasikan properti instagramUrl dengan tipe data String untuk menyimpan tautan ke profil Instagram. Tidak ada tanda koma , di ujung baris ini karena ini adalah data terakhir di dalam class tersebut.

Line 12: Kurung tutup `)` untuk mengakhiri daftar properti dari data class Friend.

DetailScreen.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul4compose.uiapp`. Package ini berfungsi menentukan lokasi file serta mengelompokkan komponen *User Interface* (UI) di dalam struktur proyek aplikasi.

Line 3 - 18: Kumpulan perintah `import` untuk memanggil class dan library yang dibutuhkan oleh fungsi pembangun UI, meliputi:

- `Image, Column, Box, Spacer, Text`: Komponen dasar UI Jetpack Compose.
- `Modifier, fillMaxSize, padding, fillMaxWidth, height`: Alat bantu untuk menyesuaikan ukuran dan tata letak komponen.
- `rememberScrollState, verticalScroll`: Fungsi untuk memberikan kemampuan *scrolling* (gulir vertikal) pada layar.
- `RoundedCornerShape, clip`: Alat bantu pemotong grafis untuk membuat sudut komponen menjadi melengkung.
- `MaterialTheme`: Pemanggil skema warna tema desain Material3 bawaan sistem.

- `Composable`: Anotasi penanda bahwa suatu fungsi akan menghasilkan elemen UI.
- `Alignment`, `ContentScale`, `FontWeight`, `dp`, `sp`: Pengaturan perataan, rasio gambar, gaya huruf, dan satuan presisi layar.
- `com.example.modul4compose.data.FriendData`: Mengimpor sumber data statis yang berisi koleksi daftar teman.

Line 20: Mendeklarasikan anotasi `@Composable` yang menandakan bahwa fungsi di bawahnya diizinkan untuk membangun dan merender elemen antarmuka pengguna pada Jetpack Compose.

Line 21: Mendeklarasikan fungsi `DetailScreen` yang menerima dua parameter masukan: `friendId` bertipe `Integer` (untuk parameter pencarian data spesifik), dan `modifier` dengan nilai bawaan `Modifier` kosong untuk keleluasaan penyesuaian tata letak komponen dari luar fungsi.

Line 23: Mendeklarasikan variabel konstan `friend` yang mencari satu objek spesifik dari koleksi data `FriendData.friends`. Pencarian (*find*) dilakukan dengan mencocokkan nilai `id` milik objek dengan `friendId` yang diteruskan sebagai parameter.

Line 25: Blok validasi kondisi `if (friend != null)` yang mengevaluasi apakah proses pencarian berhasil menemukan data. Jika data valid, maka blok kode pembentuk layar profil akan dieksekusi.

Line 26: Memanggil komponen `Column()`, yang berfungsi menyusun elemen-elemen di dalamnya secara berderet vertikal dari atas ke bawah.

Line 27: Menetapkan `modifier = modifier` dari parameter induk fungsi.

Line 28: Menambahkan argumen `.fillMaxSize()` agar komponen `Column` membentang dan menempati seluruh sisa layar yang tersedia.

Line 29: Menambahkan argumen `.padding(16.dp)` untuk memberikan jarak aman berupa ruang kosong di seluruh sisi layar sebesar 16 *density-independent pixels*.

Line 30: Menambahkan argumen `.verticalScroll(rememberScrollState())` agar seluruh konten layar dapat digulir ke atas dan ke bawah secara mandiri apabila elemen di dalamnya melebihi tinggi layar perangkat.

Line 31: Menutup kurung parameter `Column` dan membuka kurung kurawal `{` untuk memulai isi kerangka vertikal.

Line 32 - 40: Memanggil komponen `Image` untuk menampilkan foto profil dengan pengaturan spesifik:

- `painter`: Memuat gambar dari sumber daya *drawable* (`friend.photoResId`).
- `contentDescription`: Menyediakan teks alternatif (nama teman) untuk keperluan aksesibilitas pembaca layar.
- `contentScale = ContentScale.Crop`: Memastikan gambar dipotong secara proporsional agar memenuhi batas kerangkanya tanpa merusak rasio gambar.
- `modifier`: Mengatur lebar hingga penuh (`.fillMaxWidth()`), menetapkan tinggi presisi 350.dp (`.height(350.dp)`), serta memotong sudut gambar menjadi membulat dengan jari-jari 16.dp (`.clip(RoundedCornerShape(16.dp))`).

Line 42: Memanggil komponen `Spacer` yang diatur memiliki tinggi 16.dp, berfungsi sebagai jarak pemisah vertikal antara gambar profil dan teks judul di bawahnya.

Line 45 - 50: Memanggil komponen `Text` untuk mencetak nama kawan (`friend.name`). Teks ini diatur memiliki ukuran huruf 28.sp, ketebalan maksimal (`FontWeight.Bold`), serta spasi antar baris (*line height*) sebesar 32.sp.

Line 53 - 58: Memanggil komponen `Text` sekunder untuk mencetak tanggal/waktu (`friend.date`). Teks diatur sebesar 16.sp, diberikan warna yang menyesuaikan mode tema aplikasi secara dinamis (`MaterialTheme.colorScheme.onSurfaceVariant`), serta diberi jarak margin atas 4.dp dan bawah 16.dp.

Line 61 - 65: Memanggil komponen `Text` statis yang berfungsi sebagai tajuk deskripsi. Elemen ini memuat *string* "Feature:" dengan spesifikasi huruf sebesar 18.sp dan tebal (`FontWeight.Bold`).

Line 67: Memanggil elemen pembatas `Spacer` dengan tinggi kecil sebesar `4.dp` untuk memisahkan tajuk dengan paragraf isi.

Line 70 - 74: Memanggil komponen `Text` untuk menampilkan isi narasi profil (`friend.feature`). Properti ukuran huruf diatur menjadi `16.sp` dengan jarak antar baris `24.sp` untuk meningkatkan kenyamanan pembacaan (*readability*).

Line 75: Kurung kurawal tutup `}` yang mengakhiri deklarasi komponen-komponen di dalam susunan hierarki `Column`. Line 76: Blok `} else {` yang akan dieksekusi sistem sebagai bentuk penanganan kesalahan (*error handling*) apabila kondisi `friend` di awal pengecekan ternyata bernilai `null` (data dengan ID tersebut tidak ditemukan di basis data).

Line 78: Memanggil komponen wadah `Box` yang diatur membentang memenuhi layar (`.fillMaxSize()`). Atribut `contentAlignment = Alignment.Center` memastikan elemen yang berada di dalam boks diletakkan secara persis di pusat layar.

Line 79: Memanggil komponen `Text` statis untuk merender pesan kesalahan visual berbunyi "Data teman tidak ditemukan" yang akan terpampang di tengah layar kosong.

Line 80 - 82: Rangkaian penutup kurung kurawal `}` yang secara hierarkis mengakhiri komponen `Box`, blok kondisi pengecekan `else`, dan keseluruhan fungsi `@Composable DetailScreen`.

FriendItem.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul4compose.uiapp`. Package ini berfungsi untuk mengelompokkan komponen UI di dalam struktur proyek aplikasi Android.

Line 3 - 17: Kumpulan perintah `import` untuk memanggil class dan library yang dibutuhkan oleh fungsi pembangun UI antarmuka daftar teman, meliputi

- `Image`, `Column`, `Row`, `Spacer`, `Card`, `Button`, `Text`: Komponen dasar pembangun antarmuka UI Jetpack Compose.

- `RoundedCornerShape`: Alat untuk membuat sudut komponen menjadi melengkung atau membulat.
- `MaterialTheme`, `CardDefaults`, `ButtonDefaults`: Pemanggil skema warna dan gaya desain bawaan dari `Material3`.
- `Composable`: Anotasi penanda bahwa fungsi ini adalah elemen pembuat UI.
- `Modifier`: Class dasar untuk mengatur tata letak, ukuran, dan jarak komponen.
- `clip`, `ContentScale`: Alat untuk memotong bentuk gambar dan mengatur skala gambar.
- `LocalContext`: Alat untuk mendapatkan akses informasi lingkungan aplikasi saat ini.
- `painterResource`: Fungsi untuk memuat gambar dari folder sumber daya aplikasi (`drawable`).
- `FontWeight`, `TextOverflow`, `dp`, `sp`: Pengaturan gaya huruf, pemotongan teks, serta satuan presisi jarak dan teks.

`com.example.modul4compose.model.Friend`: Mengimpor cetak biru (model) struktur data `Friend` yang memuat atribut seperti nama, tanggal, gambar, dan URL.

Line 19: Mendeklarasikan anotasi `@Composable` yang menandakan bahwa fungsi di bawahnya diizinkan untuk membangun elemen antarmuka pengguna pada sistem Jetpack Compose.

Line 20 - 24: Mendeklarasikan fungsi `FriendItem` yang menerima empat parameter pembangun:

- `friend`: `Friend`: Menerima satu objek tunggal data kawan untuk ditampilkan.
- `onDetailClick`: () -> Unit: Fungsi callback (panggilan balik) yang akan dipicu ketika tombol "Detail" diklik, tanpa mengembalikan nilai.
- `onInstagramClick`: (String) -> Unit: Fungsi callback yang akan dipicu ketika tombol "Instagram" diklik dengan mengirimkan parameter bertipe String (URL Instagram).

`modifier`: `Modifier` = `Modifier`: Parameter modifier opsional untuk memberikan keleluasaan pengaturan tata letak luar komponen.

Line 25: Tanda kurung kurawal pembuka { untuk memulai isi logika dan perenderan UI dari fungsi `FriendItem`.

Line 26: Membuat variabel konstan context menggunakan `LocalContext.current` untuk menangkap konteks layar saat ini.

Line 28: Memanggil komponen wadah utama yaitu `Card()`, yang memberikan efek visual berupa kartu dengan latar belakang dan bayangan.

Line 29: Menetapkan atribut shape dengan `RoundedCornerShape(size = 16.dp)` untuk membuat keempat sudut kartu melengkung sebesar 16 density-independent pixels.

Line 30: Menetapkan atribut colors menggunakan warna standar pembungkus varian latar belakang dari tema `Material3` (`MaterialTheme.colorScheme.surfaceVariant`).

Line 31 - 32: Meneruskan argumen modifier dari parent dan menambahkan jarak margin luar (`padding`) sebesar 8.dp untuk bagian horizontal (kiri-kanan) dan vertikal (atas-bawah).

Line 33: Tanda kurung tutup dan buka kurawal `) {` yang memulai isi konten di dalam komponen `Card`.

Line 34: Memanggil kerangka tata letak `Row()`, yang berfungsi menyusun elemen-elemen di dalamnya secara berderet horizontal (kiri ke kanan).

Line 35: Memberikan jarak padding dalam sebesar 16.dp di seluruh sisi dalam `Row`.

Line 36: Menambahkan pengaturan `horizontalArrangement = Arrangement.spacedBy(space = 16.dp)` agar terdapat jarak pemisah seragam sebesar 16.dp antar elemen (antara gambar dan teks) di dalam baris tersebut.

Line 37: Tanda kurung tutup dan buka kurawal `) {` untuk memulai isi kerangka `Row`.

Line 38 - 44: Memanggil komponen `Image` untuk menampilkan foto profil dengan pengaturan spesifik:

- `painter`: Memuat berkas gambar menggunakan `painterResource` berdasarkan ID dari `friend.photoResId`.
- `contentDescription`: Menyediakan teks alternatif bernama `friend.name` untuk kebutuhan pembaca layar (`accessibility`).
- `contentScale`: Diatur ke `ContentScale.Crop` agar gambar dipotong memenuhi batas bentuknya secara proporsional.

- modifier: Memberikan ukuran absolut sebesar 100.dp (`.size(100.dp)`) dan memotong gambar menjadi bentuk bersudut lengkung 12.dp (`.clip(RoundedCornerShape(size = 12.dp))`).

Line 45: Kurung tutup `)` mengakhiri pengaturan komponen Image.

Line 47: Memanggil komponen `Column()`, yang berfungsi menyusun sisa elemen (teks dan tombol) secara berderet vertikal (atas ke bawah).

Line 48: Menambahkan atribut `.weight(weight = 1f)` pada modifier `Column`, yang memaksa kolom ini untuk menguasai dan mengisi seluruh sisa ruang lebar kosong yang tidak terpakai oleh elemen Image.

Line 49: Tanda kurung tutup dan buka kurawal `) {` untuk memulai isi kerangka `Column`.

Line 51: Memanggil sub-komponen `Row()` khusus untuk menyusun nama dan tanggal sejajar di bagian atas secara horizontal.

Line 52: Menambahkan argumen `.fillMaxWidth()` agar baris internal ini merentang selebar mungkin.

Line 53: Menambahkan pengaturan `horizontalArrangement = Arrangement.SpaceBetween` agar elemen pertama (Nama) didorong rata kiri, dan elemen terakhir (Tanggal) didorong penuh ke rata kanan.

Line 54: Tanda kurung tutup dan buka kurawal `) {` untuk memulai isi sub-komponen `Row` atas.

Line 56 - 62: Memanggil komponen `Text` untuk merender nama (`friend.name`), dengan gaya cetak tebal (`FontWeight.Bold`), ukuran huruf 16.sp, dibatasi maksimal satu baris (`maxLines = 1`), teks yang melebihi batas dipotong dengan akhiran elipsis/titik tiga (`TextOverflow.Ellipsis`), serta memakai `.weight(1f)` agar teks nama mengambil jatah sisa baris terbanyak.

Line 63: Kurung tutup `)` mengakhiri pengaturan komponen `Text` nama.

Line 65 - 68: Memanggil komponen Text sekunder untuk merender tanggal (friend.date), dengan ukuran huruf lebih kecil yaitu 10.sp, dan diberikan jarak tepi kiri (start = 8.dp) agar tidak terlalu rapat dengan teks nama.

Line 69: Kurung tutup) mengakhiri pengaturan komponen Text tanggal.

Line 71: Kurung kurawal tutup } yang mengakhiri sub-komponen Row penyusun header nama dan tanggal.

Line 73: Memanggil komponen pembatas Spacer dengan tinggi 8.dp untuk memberikan jarak vertikal antara judul dan isi deskripsi.

Line 75 - 79: Memanggil komponen Text untuk merender paragraf fitur/deskripsi (friend.feature). Pengaturannya menggunakan ukuran huruf 12.sp, dibatasi maksimal hingga 4 baris teks (maxLines = 4), dan memotong kalimat berlebih dengan tiga titik (TextOverflow.Ellipsis).

Line 80: Kurung tutup) mengakhiri komponen Text deskripsi.

Line 82: Memanggil komponen pembatas Spacer sekunder dengan tinggi 12.dp untuk memberikan jarak vertikal antara blok teks deskripsi dan susunan tombol navigasi.

Line 84: Memanggil sub-komponen Row(terbawah untuk meletakkan tombol-tombol interaksi secara sejajar horizontal.

Line 85: Menginstruksikan Row tombol untuk mengisi lebar penuh layar dengan .fillMaxWidth().

Line 86: Menetapkan jarak antar tombol (spasi di tengah) sebesar 8.dp.

Line 87: Tanda kurung tutup dan buka kurawal) { untuk memulai isi sub-komponen Row tombol.

Line 89 - 93: Memanggil komponen Button pertama (untuk Instagram) dengan menetapkan fungsi klik onInstagramClick beserta pelemparan data URL (friend.instagramUrl), memodifikasi ketebalan bantalan internal (padding) ke ukuran kompak, memakai warna tombol standar, serta menyisipkan atribut .weight(weight = 1f) agar ukuran tombol mendistribusi dirinya secara proporsional.

Line 94: Tanda kurung tutup dan buka kurawal) { memulai isi komponen di dalam tombol Instagram.

Line 95: Memanggil Text berlabel "Instagram" yang diatur berukuran 11.sp, satu baris maksimal, dengan opsi elipsis.

Line 96: Kurung kurawal tutup } mengakhiri perenderan tombol Instagram.

Line 98 - 102: Memanggil komponen Button kedua (untuk Detail) dengan menetapkan callback klik onDetailClick, modifikasi ketebalan padding yang sama dengan tombol Instagram, serta menyisipkan atribut .weight(weight = 1f) agar ukuran lebar kedua tombol tersebut seimbang (50:50).

Line 103: Tanda kurung tutup dan buka kurawal) { memulai isi komponen di dalam tombol Detail.

Line 104: Memanggil Text berlabel "Detail" yang memiliki spesifikasi identik dengan teks tombol sebelumnya.

Line 105: Kurung kurawal tutup } mengakhiri perenderan tombol Detail.

Line 106 - 110: Rangkaian penutup kurung kurawal } yang secara berurutan berfungsi untuk mengakhiri pembentukan blok kerangka hierarki, meliputi: Row tombol (Line 106), konten Column teks (Line 107), kerangka Row utama dalam kartu (Line 108), komponen Card pembungkus (Line 109), dan keseluruhan metode @Composable fun FriendItem (Line 110).

FriendListScreen.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu com.example.modul4compose.uiapp. Package ini berfungsi untuk mengelompokkan komponen antarmuka pengguna (UI) di dalam direktori struktur proyek aplikasi.

Line 3 - 13: Kumpulan perintah import untuk menyertakan library dan class yang dibutuhkan dalam membangun tata letak layar beranda, yang meliputi:

- layout.*: Kumpulan alat pengatur tata letak seperti ukuran dan jarak (padding/margin).

LazyColumn, LazyRow, items: Komponen khusus dari Jetpack Compose untuk membuat daftar yang dapat digulir (scrollable list) secara efisien karena hanya me-render elemen yang terlihat di layar.

- Text: Komponen dasar untuk mencetak tulisan.
- Composable: Anotasi wajib penanda bahwa fungsi ini adalah elemen pembuat UI.
- Modifier: Class dasar untuk mengatur dimensi dan perilaku visual komponen.
- FontWeight, dp, sp: Pengaturan gaya ketebalan huruf serta ketepatan satuan ukuran untuk jarak layar dan teks.
- com.example.modul4compose.model.Friend: Mengimpor model data yang berisi struktur atribut kawan.

Line 15: Mendeklarasikan anotasi @Composable yang menandakan bahwa fungsi di bawahnya diizinkan untuk membangun dan memodifikasi hierarki UI pada sistem Jetpack Compose.

Line 16 - 21: Mendeklarasikan fungsi FriendListScreen yang bertugas menyusun layar utama daftar teman. Fungsi ini menerima empat parameter pendukung:

- friends: List<Friend>: Menerima sekumpulan koleksi data kawan (list).
- onNavigateToDetail: (Friend) -> Unit: Fungsi callback yang menangani aksi navigasi ke halaman detail saat item diklik.
- onInstagramClick: (String) -> Unit: Fungsi callback yang menangani aksi membuka tautan Instagram saat tombol sekunder diklik.
- modifier: Modifier = Modifier: Parameter modifier opsional untuk penyesuaian tata letak dari luar.

Line 22 - 24: Memanggil komponen LazyColumn(), yaitu kerangka utama penyusun elemen secara vertikal (atas ke bawah) yang dapat digulir. Di dalamnya, modifier diinstruksikan dengan .fillMaxSize() agar daftar vertikal ini menguasai seluruh dimensi layar perangkat.

Line 25 - 32: Mendefinisikan blok tunggal item { pertama di dalam LazyColumn. Blok ini memanggil komponen Text untuk mencetak tajuk utama berjudul "Featured Friends" dengan gaya cetak tebal (FontWeight.Bold), ukuran huruf 20.sp, dan dibatasi oleh ruang padding spesifik (kiri 16.dp, atas 16.dp, bawah 8.dp).

Line 34 - 49: Mendefinisikan blok item { kedua yang di dalamnya memuat komponen LazyRow(.

- LazyRow (Line 35-38) digunakan untuk membuat daftar bergulir secara horizontal (kiri ke kanan) khusus untuk teman-teman unggulan (featured). Daftar ini diberi padding konten internal 16.dp di sisi horizontal dan jarak antar-item sebesar 16.dp menggunakan Arrangement.spacedBy.
- Di dalam LazyRow, baris 39 menggunakan items(friends.take(5)) untuk mengambil dan hanya menampilkan 5 data teman pertama dari daftar keseluruhan.
- Masing-masing dari ke-5 elemen tersebut dibungkus menggunakan komponen Box (Line 40) dengan batasan lebar tetap sebesar 350.dp agar kartu memiliki ukuran yang konsisten di layar horizontal.

Pada Line 41 - 45, komponen FriendItem dipanggil untuk me-render data ke dalam bentuk kartu secara individual, sekaligus menempelkan fungsi aksi klik detail dan aksi klik Instagram.

Line 51 - 58: Mendefinisikan blok item { ketiga yang kembali memanggil komponen Text untuk membuat tajuk pembatas dengan teks "All Friends". Format dan gayanya identik dengan tajuk sebelumnya, namun dengan nilai padding atas (top = 24.dp) yang lebih besar untuk memberikan jarak pernapasan visual dari elemen LazyRow di atasnya.

Line 60: Memanggil fungsi items(friends) { friend -> (tanpa blok pembungkus item tunggal). Pemanggilan fungsi jamak ini akan me-looping (mengulang) seluruh data kawan yang ada di dalam variabel koleksi friends untuk diletakkan berderet ke bawah di dalam LazyColumn.

Line 61 - 66: Memanggil komponen FriendItem untuk me-render sisa keseluruhan data teman dalam bentuk susunan kartu vertikal, serta memasukkan pemetaan callback onDetailClick dan onInstagramClick dengan logika yang sama persis dengan yang ada di daftar horizontal.

Line 67 - 68: Merupakan susunan tanda kurung kurawal tutup } berurutan yang menandakan berakhirnya pendefinisian ruang lingkup hierarki LazyColumn dan penyelesaian kerangka fungsi FriendListScreen secara keseluruhan.

AppNavigation.kt

Line 1: Deklarasi package `com.example.modul4compose.uiapp` yang menetapkan identitas dan ruang lingkup hierarki *folder* di mana file komponen UI navigasi ini disimpan dalam struktur proyek.

Line 3 - 16: Instruksi import untuk memanggil antarmuka pemrograman aplikasi (API) dan library yang diperlukan:

- `android.content.Intent` dan `android.net.Uri`: Kelas inti Android yang memfasilitasi komunikasi antar-aplikasi, khusus di sini digunakan untuk membangun `Explicit Intent` yang mengarahkan pembukaan sebuah tautan (URL) ke peramban atau aplikasi eksternal.
- `androidx.compose.runtime.Composable`: Anotasi penanda bahwa fungsi bertindak sebagai pembangun antarmuka deklaratif pada Jetpack Compose.
- `androidx.compose.runtime.collectAsState` dan `getValue`: Utilitas reaktif yang mengonversi pemancaran data `StateFlow` dari `ViewModel` menjadi status mutakhir yang dapat diawasi langsung oleh UI.
- `androidx.compose.ui.Modifier`: Objek fundamental untuk mendeklarasikan properti tata letak visual pada suatu elemen.
- `androidx.compose.ui.platform.LocalContext`: Penyedia akses ke konteks hierarki Android saat ini, yang secara mutlak diperlukan untuk mengeksekusi operasi tingkat sistem seperti `Intent`.
- `androidx.lifecycle.viewmodel.compose.viewModel`: Utilitas injeksi yang bertanggung jawab menautkan instansiasi `ViewModel` ke dalam daur hidup (lifecycle) komponen Compose.
- `androidx.navigation.*`: Memuat seluruh kelas tulang punggung komponen Navigasi Jetpack (`NavHost`, `route composable`, `rememberNavController`, `NavType`, `navArgument`) yang memfasilitasi transisi antar-layar.
- `timber.log.Timber`: Modul pihak ketiga pendukung diagnostik (debugging) yang terotomatisasi untuk mencatat riwayat pemanggilan fungsi ke dalam konsol Logcat.

Line 18: Penggunaan anotasi `@Composable` untuk menetapkan fungsi ini sebagai pembentuk struktur dan visual UI.

Line 19: Mendeklarasikan fungsi tata letak utama navigasi, `AppNavigation`, yang mengakomodasi penerimaan parameter opsional modifier dengan nilai rujukan awal sebuah elemen Modifier kosong.

Line 20: Melakukan inisialisasi variabel status `navController` melalui pemanggilan fungsi pembantu `rememberNavController()`. Objek persisten ini akan menjadi orkestrator tunggal untuk mengarahkan pengguna melintasi jalur aplikasi dan menjaga rekaman tumpukan layar (back stack).

Line 23 - 25: Menginstansiasi dan menyuntikkan variabel `viewModel` (yang memiliki tipe data `FriendViewModel`) melalui pemanggilan `factory method` kustom `FriendViewModelFactory(name = "Fathi")`. Pendekatan ini melempar data inisialisasi secara manual agar `ViewModel` memiliki parameter identitas string yang dibutuhkan sejak fase pembentukannya.

Line 27 - 31: Mengonfigurasi kanvas penampung navigasi utama (`NavHost`):

- Menautkan `navController` yang telah diinisialisasi agar menjadi mesin penggeraknya.
- Menginstruksikan rute pembuka dengan parameter `startDestination = "list_screen"`.
- Menerapkan turunan parameter modifikasi tata letak (`modifier = modifier`) dari induk (seperti pengaturan jarak/padding).
- Membuka blok `lambda` menggunakan `{` untuk mendefinisikan graf rute secara berurutan.

Line 33: Mendaftarkan halaman (simpul) pertama di dalam graf navigasi menggunakan perintah `composable(route = "list_screen") {`. Apabila pengguna belum berada di layar mana pun, rute ini akan dieksekusi pertama kali.

Line 34: Menginisiasi deklarasi variabel proksi bernama `friends` yang didelegasikan (by) ke nilai mutakhir dari properti `friendsState` (milik `viewModel`) melalui perantara `.collectAsState()`. Dengan begini, perombakan data dalam kumpulan teman di memori logis akan memicu perenderan ulang layar.

Line 35: Menginstruksikan ekstraksi dan penyimpanan referensi operasional dari lingkungan aplikasi (Context) ke dalam variabel konstan context menggunakan LocalContext.current.

Line 37 - 53: Melakukan perenderan visual tampilan daftar kawan melalui injeksi pemanggilan FriendListScreen(. Di dalamnya, didefinisikan perilaku (behaviour) dari setiap paramater dinamis yang dituntut oleh fungsi tersebut:

- Line 38: Menyodorkan nilai mutakhir variabel status (friends) untuk memetakan visual list teman.
- Line 39 - 44: Mendefinisikan implementasi lambdal callback navigasi (onNavigateToDetail). Saat dipicu, algoritma ini menugaskan ViewModel mencatat aksi tersebut (melalui pemanggilan fungsi log viewModel.onFriendClicked), dilanjutkan dengan memberikan perintah ke navController untuk menjejalkan layar baru di atas stack menuju rute detail_screen/ yang disisipi dengan ID kawan (friend.id).
- Line 45 - 52: Mendefinisikan implementasi lambdal callback eksternal (onInstagramClick) yang menangkap argumen url. Ketika dipicu, sistem mencatat riwayat penekanan tombol menggunakan Timber.d(), mendirikan instruksi sistem Intent baru berbasis aksi pandangan tayang (ACTION_VIEW) terhadap objek URL yang telah diparsing, dan pada puncaknya, mengeksekusi peluncuran aktivitas luar menggunakan bantuan modul context.startActivity(intent).

Line 54: Menutup blok lambda halaman pembuka (list_screen).

Line 57 - 64: Mendaftarkan halaman kedua bertipe dinamis di dalam graf navigasi:

- Line 57 - 58: Meregistrasi identitas pola rute "detail_screen/{friendId}". String berpola kurung kurawal ini memberitahu mesin navigasi bahwa ada fragmen jalur yang dapat disubstitusi.
- Line 59: Menerapkan parameter aturan (contract) bahwa data masuk (argumen friendId) wajib dimengerti sebagai sekumpulan angka bulat presisi standar (NavType.IntType).
- Line 60: Membuka ekspresi blok perenderan yang disertai penangkapan elemen rekam penjelajahan (back stack entry).

- Line 61: Melakukan ekstraksi variabel secara aman (?) dari bundle argumen `backStackEntry`. Apabila key "friendId" ditemukan, nilainya akan dikonversi ke variabel `id`; namun jika gagal atau null, ditetapkan pengaman bawaan (fallback) berupa angka 0 lewat operator Elvis (`?:`).
- Line 62: Melakukan injeksi hasil konversi variabel `id` tadi ke dalam panggilan perenderan layar detail kawan, `DetailScreen()`.

Line 63: Menutup blok *lambda* deklarasi rute halaman detail.

Line 65: Tanda kurung kurawal penutup `}` untuk mengakhiri komposisi keseluruhan bagan hierarki penjelajahan (`NavHost`) serta fungsi `AppNavigation` itu sendiri.

FriendViewModel.kt

Line 1: Deklarasi package file Kotlin yaitu `com.example.modul4compose.uiapp`. Package ini berfungsi untuk mengelompokkan kelas-kelas yang berkaitan dengan antarmuka pengguna dan logika presentasi di dalam proyek.

Line 3 - 10: Kumpulan perintah `import` untuk memanggil antarmuka pemrograman (API) yang dibutuhkan:

- `androidx.lifecycle.ViewModel` dan `ViewModelProvider`: Kelas dasar untuk menerapkan pola arsitektur MVVM (Model-View-ViewModel) guna menjaga kelangsungan data selama siklus hidup antarmuka.
- `com.example.modul4compose.data.FriendData` dan `model.Friend`: Mengimpor sumber data mentah statis dan kerangka cetak biru model datanya.
- `kotlinx.coroutines.flow.*`: Mengimpor `MutableStateFlow`, `StateFlow`, dan `asStateFlow` untuk mengelola distribusi aliran data reaktif (observable state) secara asinkron.
- `timber.log.Timber`: Library pihak ketiga untuk mencetak aktivitas sistem (debugging) ke konsol Logcat.

Line 12: Mendeklarasikan kelas FriendViewModel yang mewarisi fungsionalitas dari kelas ViewModel(). Kelas ini memiliki konstruktor utama yang mewajibkan penyisipan satu parameter konstan bernama userName dengan tipe data String.

Line 15: Mendeklarasikan variabel privat _friendsState menggunakan fungsionalitas MutableStateFlow yang diinisialisasi dengan senarai kosong (emptyList()). Variabel ini bertindak sebagai penyimpan status internal yang sifatnya dapat diubah (mutable) mutlak dari dalam kelas itu sendiri.

Line 16: Mendeklarasikan variabel publik friendsState bertipe StateFlow<List<Friend>> dengan mendelegasikan pembacaan nilainya ke variabel _friendsState.asStateFlow(). Pendekatan ini menerapkan prinsip enkapsulasi agar komponen UI luar hanya dapat mengawasi data secara read-only tanpa berwenang mengubah isinya.

Line 18: Membuka blok init {, yang mana setiap instruksi di dalamnya akan dieksekusi secara otomatis oleh sistem sesaat setelah objek kelas FriendViewModel berhasil dibentuk.

Line 19: Melakukan pemanggilan fungsi internal loadFriends() untuk memulai proses pemuatan data teman.

Line 20: Mengeksekusi pencetakan pesan log debug menggunakan fungsi Timber.d(). Baris ini bertugas merekam status inisialisasi awal ViewModel sekaligus mencetak nilai dari parameter tangkapan userName.

Line 21: Kurung kurawal tutup } untuk mengakhiri eksekusi rentetan instruksi di dalam blok inisialisasi awal.

Line 23: Mendeklarasikan fungsi loadFriends() menggunakan access modifier privat (private fun), sehingga fungsi pengambilan data ini dilindungi dan hanya diizinkan untuk dipicu dari lingkup internal kelas ViewModel ini saja.

Line 24: Membuat variabel lokal data untuk mengalokasikan dan mengambil rujukan daftar kawan dari pangkalan data objek FriendData.friends.

Line 25: Melakukan pembaruan terhadap aliran data reaktif dengan cara mendistribusikan isi variabel data ke dalam properti .value milik _friendsState. Aksi ini akan seketika memicu perenderan ulang antarmuka UI yang sedang mengawasi StateFlow tersebut.

Line 26: Mencetak log debug menggunakan Timber.d() yang merekam jejak operasi pemuatan, lengkap dengan perhitungan otomatis terhadap jumlah data yang dimuat ke dalam list (data.size).

Line 27: Kurung kurawal tutup } untuk menyatakan berakhirnya pendefinisian logika dari fungsi loadFriends().

Line 30: Mendeklarasikan fungsi operasional publik onFriendClicked yang menerima masukan satu parameter utuh berupa objek Friend. Fungsi ini dirancang sebagai pendengar (handler) logis terhadap interaksi klik pengguna di antarmuka aplikasi.

Line 31: Memicu cetakan log pertama melalui Timber.d() yang melaporkan peristiwa ditekan-nya tombol rincian (Detail), yang secara spesifik mencantumkan nama subjek kawan yang bersangkutan (friend.name).

Line 32: Memicu cetakan log kedua yang mengonfirmasi peralihan fase transisi layar, serta membedah dan merekam keseluruhan variabel objek friend yang ditransfer.

Line 33: Kurung kurawal tutup } untuk mengakhiri pendefinisian fungsionalitas blok onFriendClicked.

Line 34: Tanda kurung kurawal tutup } yang menandakan batas akhir dari definisi hierarki pada kelas FriendViewModel.

Line 37: Mendeklarasikan kelas FriendViewModelFactory yang mewarisi antarmuka spesifik ViewModelProvider.Factory. Kelas ini dirancang khusus untuk memfasilitasi pembentukan objek FriendViewModel yang memiliki parameter konstruktor khusus, mengingat sistem inisialisasi default Android tidak mendukung perlakuan terhadap ViewModel berparameter.

Line 38: Melakukan konstruksi ulang (override) secara eksplisit pada fungsi generik create(). Fungsi ini mewajibkan pendiktean alur mengenai tata cara sistem dalam membentuk entitas kelas ViewModel yang baru.

Line 39: Membuka susunan percabangan if untuk memvalidasi secara logis kelayakan apakah kelas yang diinstruksikan oleh sistem (modelClass) ekuivalen dan berasal dari cetakan kelas FriendViewModel.

Line 40: Menempelkan instruksi anotasi `@Suppress("UNCHECKED_CAST")` agar mesin kompilasi mengabaikan peringatan keamanan mengenai konversi tipe, berhubung validasi keamanan sudah ditangani langsung oleh baris percabangan logika di atasnya.

Line 41: Membangun (instantiate) dan mengembalikan objek `FriendViewModel` sembari menginjeksi argumen nama, yang selanjutnya hasil balikan tersebut dikonversi secara mutlak menuju tipe luaran berjenis generik `T`.

Line 42: Tanda kurung kurawal tutup `}` yang membatasi ruang lingkup operasional dari penyelesaian pengkondisian percabangan `if`.

Line 43: Menginisiasi skenario melempar pengecualian fatal (`throw exception`) berjenis `IllegalArgumentException`. Baris perlindungan ini akan mengakibatkan penghentian aplikasi secara paksa seraya menampilkan pesan "Unknown ViewModel class" manakala fungsi pabrik (factory) disodori tipe `ViewModel` yang tidak kompatibel dengannya.

Line 44: Kurung kurawal tutup `}` yang memberitahukan penyelesaian deklarasi metode spesifik `create()`.

Line 45: Tanda kurung kurawal tutup `}` terakhir sebagai segel definitif untuk menutup secara menyeluruh fungsionalitas ruang lingkup pada kelas `FriendViewModelFactory`.

MainApplication.kt

Line 1: Mendeklarasikan package file Kotlin yaitu `com.example.modul4compose`. Ini berfungsi sebagai identitas fundamental dan penunjuk lokasi struktural file di dalam hierarki direktori proyek aplikasi Android.

Line 3 - 4: Mengimpor kelas dan pustaka (library) pendukung tingkat sistem yang diwajibkan oleh file ini, meliputi:

- `android.app.Application`: Kelas dasar (base class) bawaan sistem Android yang mencakup ruang lingkup status global aplikasi.
- `timber.log.Timber`: Modul pustaka pihak ketiga yang berperan sebagai utilitas pencatatan log terotomatisasi untuk proses pelacakan masalah (debugging).

Line 6: Mendeklarasikan kelas `MainApplication` yang secara eksplisit mewarisi fungsionalitas dari kelas induk `Application()`. Kelas ini dikonfigurasi untuk menjadi titik mula (entry point) instansiasi tingkat aplikasi yang akan dieksekusi oleh sistem operasi secara absolut sebelum komponen antarmuka atau aktivitas (`Activity`) apa pun dimuat ke layar.

Line 7: Melakukan deklarasi penimpaan (override) pada metode `onCreate()`. Tahapan siklus hidup aplikasi ini akan dipanggil oleh sistem Android tepat satu kali pada saat kerangka aplikasi pertama kali dilokasikan ke dalam memori perangkat.

Line 8: Mengeksekusi fungsi `super.onCreate()` untuk mendelegasikan instruksi kepada kelas induk, guna memastikan sistem Android menyelesaikan seluruh inisialisasi lingkungan (environment) serta alokasi memori dasar tingkat aplikasi dengan benar.

Line 10: Mendeklarasikan variabel konstan `isDebuggable` yang menampung nilai boolean hasil dari evaluasi mode kompilasi. Sistem mengeksekusi operasi logika bitwise `and` antara bendera aplikasi yang aktif saat ini (`applicationInfo.flags`) dengan bendera statis absolut milik Android (`ApplicationInfo.FLAG_DEBUGGABLE`) guna memverifikasi secara pasti apakah kerangka kerja sedang beroperasi dalam mode pengembangan (debug mode) atau mode rilis (production).

Line 11: Membuka struktur percabangan evaluasi `if (isDebuggable) {`. Blok kondisional ini bertindak sebagai gerbang pengaman logis yang memastikan bahwa instruksi pelacakan di dalamnya tidak akan pernah aktif jika aplikasi telah dipublikasikan secara komersial.

Line 12: Menjalankan eksekusi penanaman konfigurasi (planting) instans pohon pencatatan log milik Timber dengan memanggil `Timber.plant(Timber.DebugTree())`. Konfigurasi tunggal ini menghidupkan mesin logger yang akan menangkap dan menyiarkan seluruh jejak variabel serta pesan sistem langsung ke konsol Logcat selama fase pengembangan.

Line 13: Tanda kurung kurawal tutup `}` yang mendefinisikan batas akhir dari ruang lingkup eksekusi pada kondisi percabangan pengecekan debug.

Line 14: Tanda kurung kurawal tutup `}` untuk mengakhiri rentetan instruksi fungsional dari siklus metode `onCreate()`.

Line 15: Tanda kurung kurawal tutup } final yang menyegel dan mengakhiri seluruh susunan hierarki struktural dari kelas MainApplication.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.

A. Penjelasan

1 Pengertian Application Class

Dalam arsitektur aplikasi Android, `Application` class adalah kelas dasar (*base class*) yang merepresentasikan dan mengelola status global dari sebuah aplikasi. Kelas ini adalah komponen pertama yang diinisialisasi oleh sistem operasi Android—bahkan sebelum komponen antarmuka seperti `Activity`, `Fragment`, atau `Service` dimuat ke dalam memori. Berdasarkan siklus hidupnya, objek dari `Application` class akan terus hidup selama proses aplikasi masih berjalan di latar depan maupun latar belakang.

2 Fungsi Utama Application Class

Penggunaan kelas turunan dari `Application` (seperti `MainApplication` pada proyek praktikum ini) memiliki beberapa fungsi krusial:

- **Inisialisasi Satu Kali (One-time Initialization):** Karena metode `onCreate()` pada `Application` class hanya dipanggil tepat satu kali saat aplikasi pertama kali dijalankan, tempat ini sangat ideal untuk melakukan inisialisasi pustaka pihak ketiga (library) tingkat global yang berat, seperti pustaka `Dependency Injection` (Dagger/Hilt), basis data lokal (Room), atau pustaka debugging (Timber).
- **Pengelolaan Status Global (Global State Management):** Menjadi wadah penyimpanan variabel atau konfigurasi yang harus dapat diakses secara persisten oleh seluruh komponen aplikasi tanpa terpengaruh oleh siklus hidup `Activity` (seperti saat terjadi rotasi layar).
- **Pemantauan Siklus Hidup Global:** Menyediakan metode seperti `registerActivityLifecycleCallbacks` untuk mendeteksi status aktivitas (kapan aplikasi masuk ke background atau foreground) secara terpusat tanpa harus menulis kode berulang di setiap `Activity`.

3 Implementasi dan Kaitan dengan Proses Debugging

Sesuai dengan prinsip arsitektur yang baik, fitur-fitur pemantauan (*debugging*) dari modul sebelumnya wajib dipertahankan dan diintegrasikan secara efisien. Pada proyek ini, pemusatan fitur *debugging* diimplementasikan dengan menanamkan pustaka Timber di dalam `MainApplication`.

Contoh Debugging di Android Studio melalui Application Class:

Tabel 22 Contoh Debugging

1	<code>class MainApplication : Application() {</code>
2	<code> override fun onCreate() {</code>
3	<code> super.onCreate()</code>
4	<code></code>
5	<code> // Memastikan fitur debugging (Timber) dari</code>
	<code>modul sebelumnya tetap dipertahankan</code>
6	<code> // dan diinisialisasi secara global hanya</code>
	<code>pada mode Debug.</code>
7	<code> val isDebuggable = (applicationInfo.flags</code>
	<code>and</code>
	<code>android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE)</code>
	<code>!= 0</code>
8	<code> if (isDebuggable) {</code>
9	<code> Timber.plant(Timber.DebugTree())</code>
10	<code> }</code>
11	<code> }</code>
12	<code>}</code>

```
class MainApplication : Application() { override fun onCreate() { super.onCreate()
// Memastikan fitur debugging (Timber) dari modul sebelumnya tetap dipertahankan //
dan diinisialisasi secara global hanya pada mode Debug. val isDebuggable =
(applicationInfo.flags                                     and
```

```
android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE) != 0 if (isDebuggable)
{ Timber.plant(Timber.DebugTree()) } }
```

REFERENSI

- Android Developers. (n.d.). *Application*. Android Developers Reference.
<https://developer.android.com/reference/android/app/Application>
- Android Developers. (n.d.). *Guide to app architecture*. Android Developers.
<https://developer.android.com/topic/architecture>
- R. Nunkesser, “Choosing a Global Architecture for Mobile Applications”.

TAUTAN GIT

<https://github.com/ZulFath1/PraktikumMobile.git>